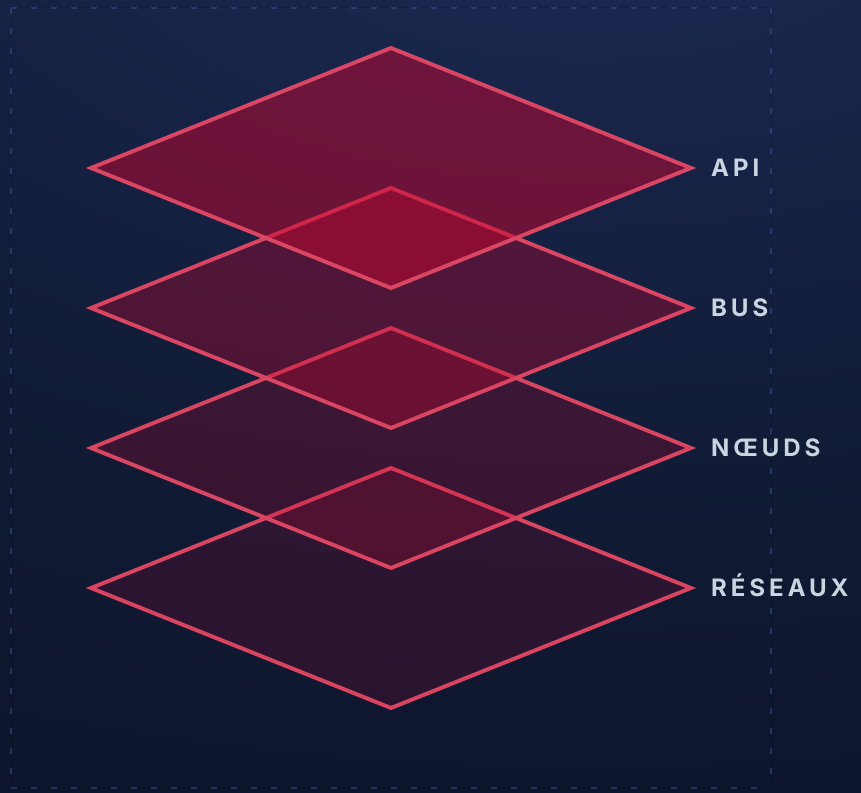
COMPRENDRE OPENSTACK · MODULE 2

Architecture interne d'OpenStack

Rôle des nœuds, communication entre composants, segmentation réseau du plan de contrôle

- Contrôle · calcul · passerelle · stockage
- API REST et bus de messages
- Réseaux du plan de contrôle



Version de référence : OpenStack 2026.2 « Hibiscus » (30 septembre 2026) · réseau de référence : ML2/OVN

INTRODUCTION

Objectif et déroulé

Objectif : savoir lire un déploiement OpenStack de l'intérieur — quel processus tourne sur quel nœud, par quel protocole il dialogue avec les autres, et sur quel réseau ce dialogue circule.

À l'issue de ce module, vous saurez :

- Décrire le contenu d'un nœud de **contrôle**, de **calcul**, de **passerelle** et de **stockage**, processus par processus.
- Distinguer les quatre protocoles internes : **REST**, **AMQP (RPC)**, **SQL** et **OVSDB**, et dire qui utilise lequel.
- Suivre une requête API de la VIP jusqu'à la base, puis un appel RPC de file en file.
- Expliquer les échanges Nova ↔ Neutron et Nova ↔ Cinder.
- Nommer les **réseaux** d'un déploiement, dire quels nœuds y sont raccordés et quels ports y circulent.
- Relier une panne à l'étage qui la provoque.

1 · Rôle des nœuds

Plan de contrôle et plan de données, anatomie de chaque type de nœud, répartition des services, topologies, comportement en cas de panne.

2 · Communication entre composants

Requête API, points d'accès, jetons de service, AMQP et oslo.messaging, conductor, base de données, RabbitMQ, OVN, séquences inter-services, diagnostic.

3 · Segmentation réseau du plan de contrôle

Domaines de sécurité, réseaux et VLAN, VIP interne et externe, matrice de flux, chiffrement, câblage, MTU, traduction en configuration.

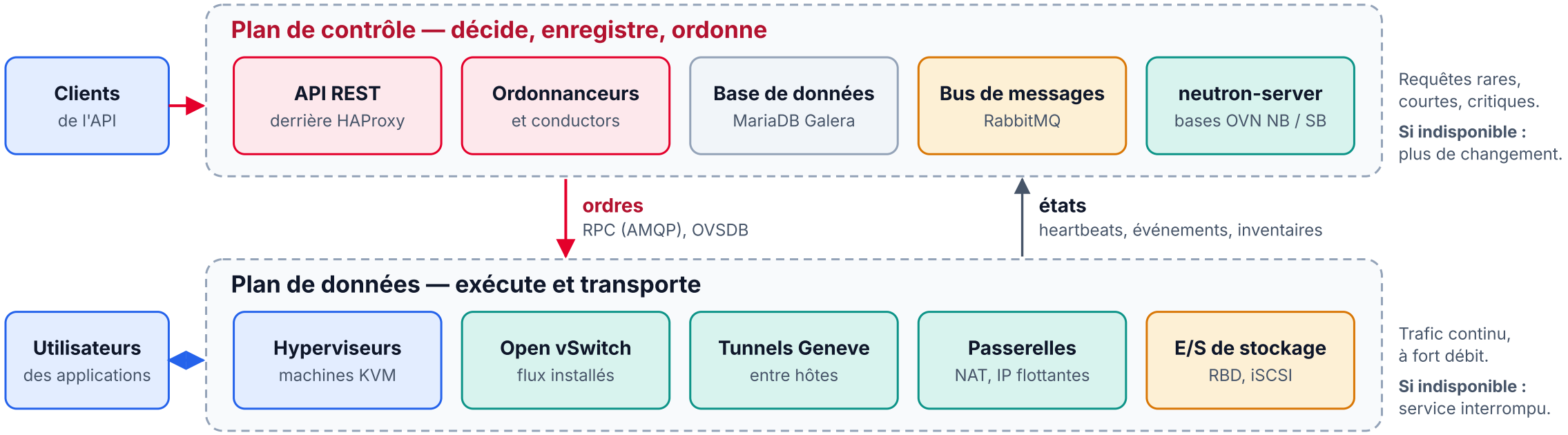
01

Rôle des nœuds

Un « nœud » est un serveur physique auquel on attribue un rôle. Ce rôle détermine les processus qu'il héberge, les réseaux auxquels il est raccordé et ce que l'on perd s'il tombe.

- Plan de contrôle, plan de données, plan de gestion
- Anatomie d'un nœud de contrôle
- Passerelles réseau et nœuds de stockage
- Les familles de nœuds
- Anatomie d'un nœud de calcul
- Répartition des services, topologies, pannes

Plan de contrôle, plan de données : deux circuits distincts



Plan de contrôle

Tout ce qui reçoit une demande, la valide, choisit où la réaliser et en conserve l'état. Il vit sur les **nœuds de contrôle** et ne voit jamais passer le trafic des instances.

Plan de données

Ce qui fait tourner les instances et achemine leurs paquets et leurs E/S. Il vit sur les **nœuds de calcul, de passerelle et de stockage** et continue de fonctionner sans le plan de contrôle.

Plan de gestion

Ce qui sert à exploiter le cloud lui-même : déploiement, accès SSH, supervision, journaux, pilotage matériel (IPMI). Il doit rester joignable quand tout le reste est en panne.

Les familles de nœuds

Contrôle

API, ordonnanceurs, conductors, base de données, bus de messages, bases OVN, HAProxy.

Nombre : 3 (toujours impair)
Perte : plus d'opérations si le quorum disparaît

Calcul

Hyperviseur KVM, nova-compute, ovn-controller, connecteurs de stockage. Héberge les instances.

Nombre : de 1 à plusieurs milliers
Perte : les instances de cet hôte

Passerelle réseau

Sortie vers les réseaux externes, NAT, IP flottantes. Souvent fusionnée avec les nœuds de contrôle.

Nombre : 2 au minimum
Perte : bascule du trafic nord-sud

Stockage

Ceph (MON, MGR, OSD) ou Swift, ou cinder-volume avec LVM. Porte volumes, images et objets.

Nombre : 3 au minimum pour Ceph
Perte : cluster dégradé, sans coupure

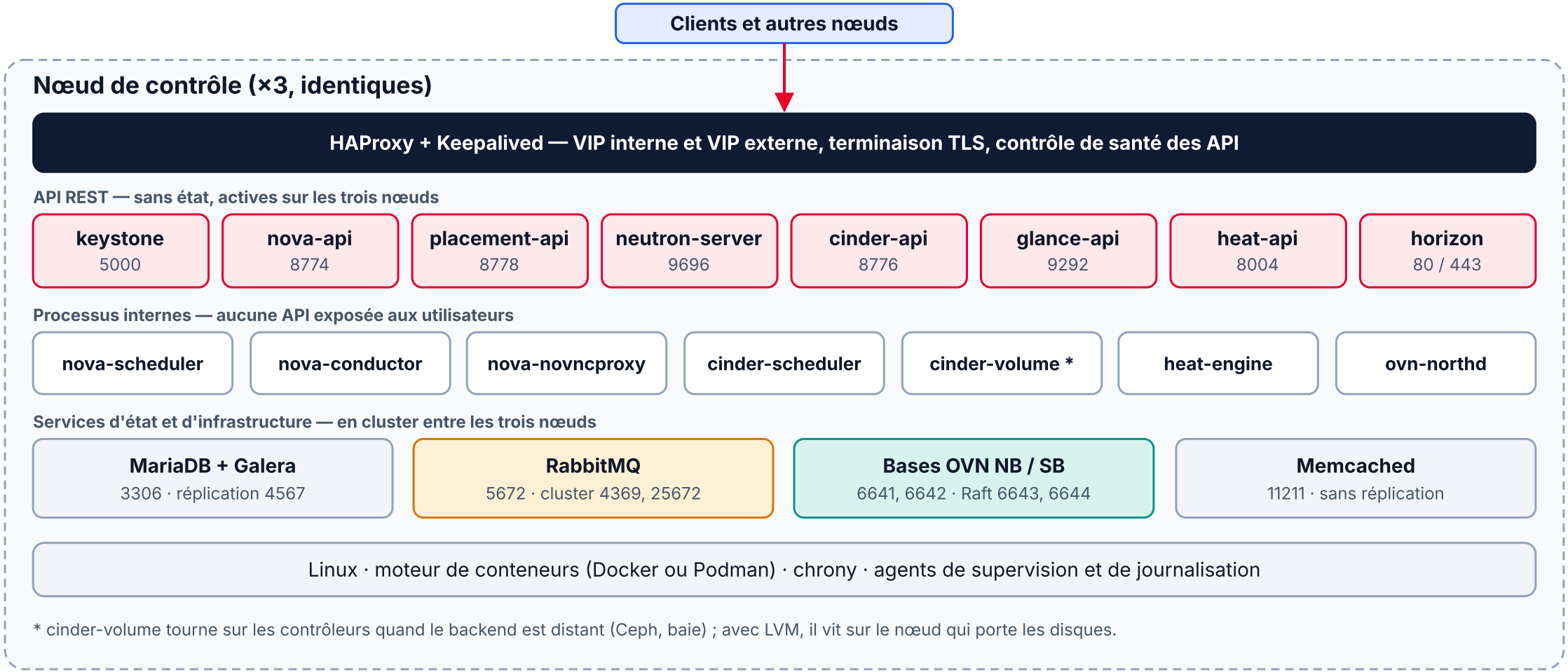
Annexes

Déploiement, bastion, supervision, journaux. Hors du chemin des requêtes des utilisateurs.

Nombre : 1 à 3
Perte : plus de visibilité ni de changement

> Un rôle est **logique** : on peut en cumuler plusieurs sur une même machine (contrôle + passerelle, calcul + OSD en hyperconvergé) ou au contraire les éclater (base et bus sur des nœuds dédiés). Ce qui compte est de savoir quels processus et quels réseaux chaque machine porte.

Anatomie d'un nœud de contrôle

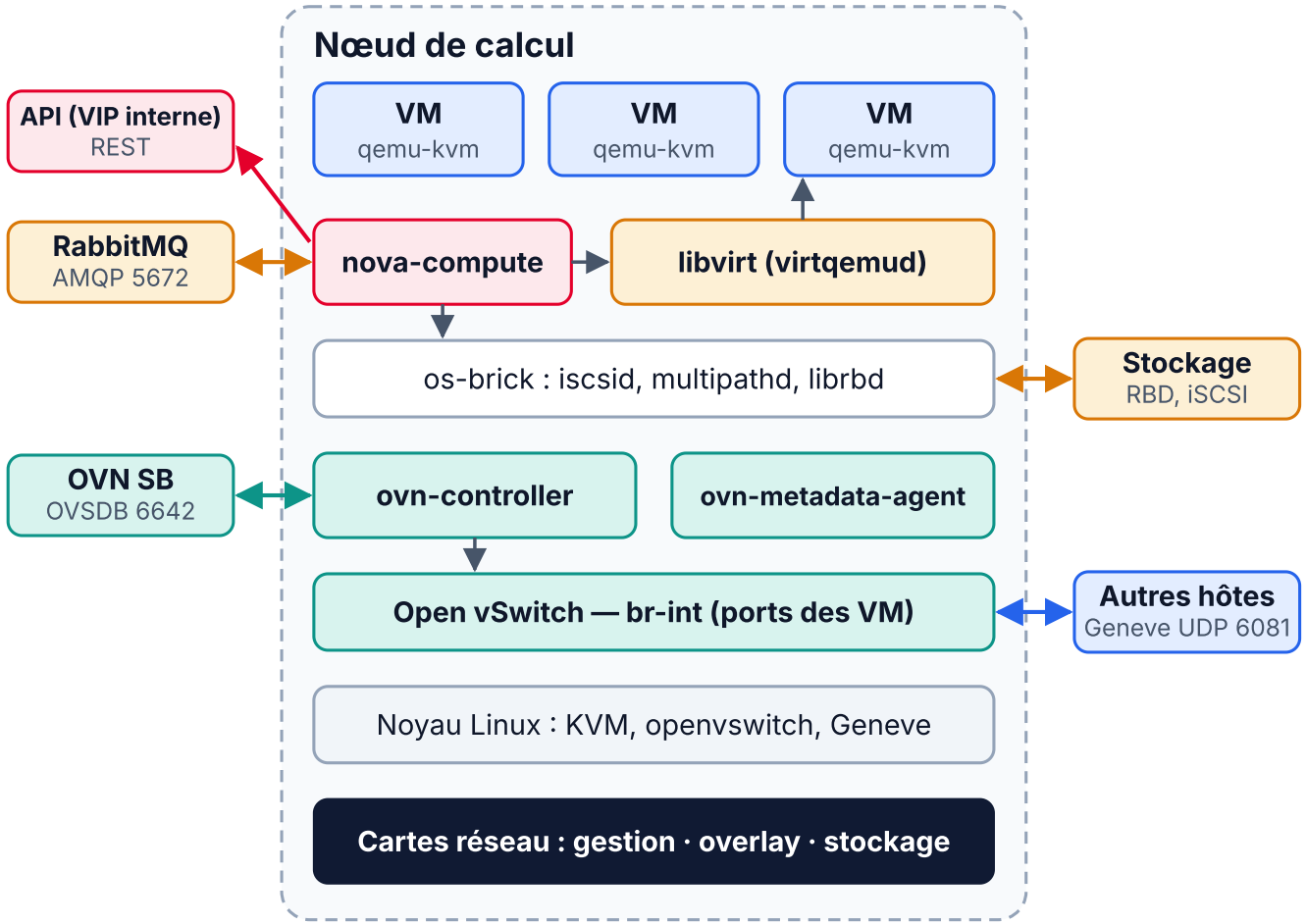


Nœud de contrôle : état, redondance et mode de fonctionnement

Composant	Porte un état ?	Mode en haute disponibilité	Mécanisme	À surveiller
API (nova, neutron, cinder…)	Non	Actif / actif	HAProxy répartit, retire un nœud qui échoue au contrôle de santé	Nombre de workers WSGI, temps de réponse
Scheduler, conductor, engine	Non	Actif / actif	Consommateurs concurrents d'une même file RabbitMQ	Profondeur des files
cinder-volume	Non, mais exclusif	Actif / passif ou actif / actif selon le pilote	Verrous distribués (coordination tooz : etcd, Redis)	Un seul actif par backend sans coordination
ovn-northd	Non	Actif / passif	Verrou pris dans la base Southbound	Un seul actif à la fois
Bases OVN NB / SB	Oui	Cluster Raft, 3 membres	Un leader élu ; quorum de 2	Leader, retard de réplication
MariaDB Galera	Oui	Multi-maître synchrone, 3 membres	Écritures dirigées vers un seul nœud par HAProxy ; quorum de 2	`wsrep_cluster_size`, état `Synced`
RabbitMQ	Oui (files)	Cluster, files à quorum (Raft)	Les clients listent les trois nœuds et se reconnectent seuls	Partitions, files sans consommateur
Memcached	Cache seulement	Instances indépendantes	Les clients connaissent la liste complète des serveurs	Perte = ralentissement, jamais perte de données
HAProxy + Keepalived	Non	Actif / passif pour les VIP	VRRP : un seul nœud porte les adresses virtuelles	Bascule de VIP, double maître

Pourquoi trois nœuds : Galera, RabbitMQ et OVN décident à la majorité. Avec trois membres, on tolère la perte d'un nœud ; avec deux, la moindre coupure réseau bloque tout le monde (« split-brain » évité au prix de l'arrêt).

Anatomie d'un nœud de calcul



Ce qu'il fait

- **nova-compute** reçoit ses ordres par RPC et les traduit en appels libvirt.
- **libvirt** lance un processus `qemu-kvm` par instance.
- **os-brick** connecte les volumes (RBD, iSCSI, FC, NVMe-oF).
- **ovn-controller** programme les flux d'Open vSwitch à partir de la base Southbound.
- **ovn-metadata-agent** relaie les requêtes `169.254.169.254` des instances.

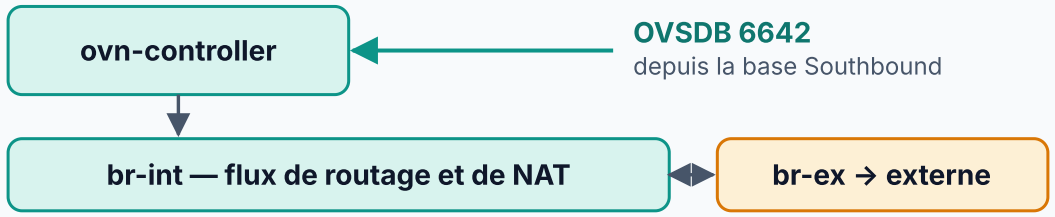
Ce qu'il ne fait pas

- Aucune API exposée aux utilisateurs, aucun accès à la base : voir le motif « conductor », slide 29.

Ports entrants : 22 (SSH), 16509 / 16514 (libvirt), 49152–49215 (migration QEMU), 5900–5999 (consoles VNC), UDP 6081 (Geneve).

Passerelle réseau : chassis OVN ou nœud à agents

ML2/OVN — chassis passerelle



- Aucun agent Neutron ni espace de noms : tout est fait par des flux OVS.
- Rôle activé par l'option ovn-cms-options=enable-chassis-as-gw.
- Chaque routeur est planifié sur plusieurs chassis, par priorité ; BFD détecte la panne et le suivant prend le relais.
- Avec le routage distribué, les IP flottantes sortent directement par l'hyperviseur ; seul le SNAT passe par la passerelle.

ML2/Open vSwitch — nœud réseau à agents

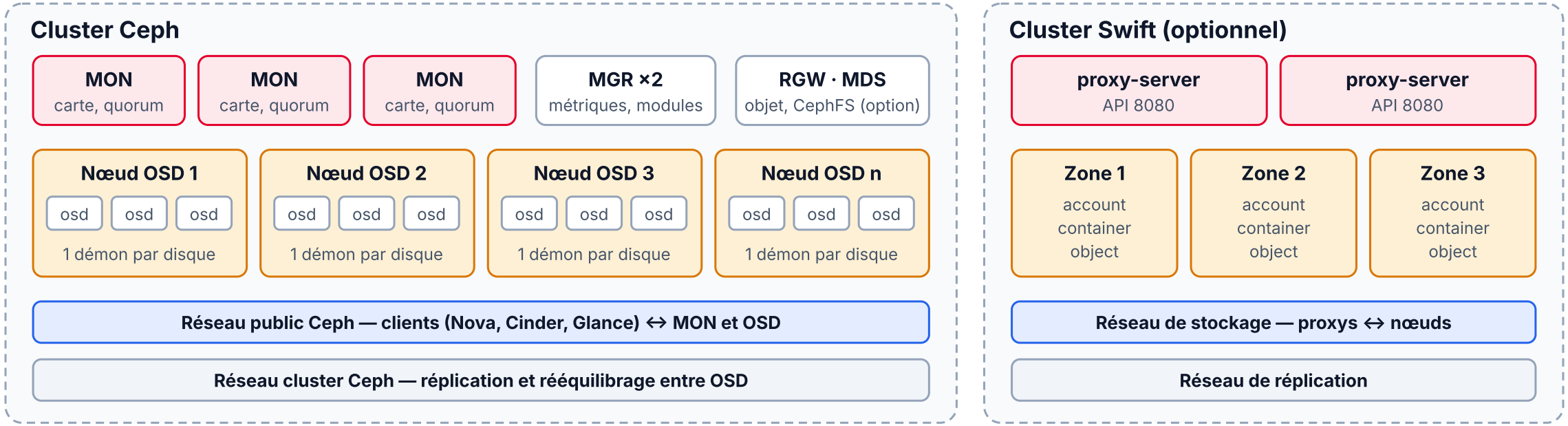


- Agents pilotés par neutron-server à travers RabbitMQ (RPC).
- Agent L3 : un espace de noms qrouter-… par routeur (iptables, keepalived).
- Agent DHCP : un espace de noms qdhcp-… et un dnsmasq par réseau.
- Nœud critique : sans routage distribué (DVR), tout le trafic nord-sud et inter-réseaux y transite.

Dans les deux cas, la passerelle est le point où un réseau de projet (overlay) rejoint un réseau physique. Elle doit donc être raccordée à la fois au réseau **overlay** et au réseau **externe**.

Choix courant : fusionner ce rôle avec les nœuds de contrôle pour les petits clouds, le dédier dès que le débit nord-sud devient important ou que l'on veut isoler la charge.

Nœuds de stockage : Ceph et Swift



Service OpenStack	Ce qu'il place dans Ceph	Pool habituel
Glance	Images (format raw conseillé)	`images`
Cinder	Volumes et sauvegardes	`volumes`, `backups`
Nova	Disques éphémères des instances	`vms`

- Les **MON** tiennent la carte du cluster : trois, pour le quorum ; souvent hébergés sur les contrôleurs dans un petit cloud.
- Les clients parlent **directement aux OSD** : les nœuds de calcul doivent donc joindre le réseau public Ceph.
- Avec **LVM**, pas de cluster : `cinder-volume` tourne sur le nœud qui porte les disques et exporte en iSCSI.

Les nœuds annexes : indispensables à l'exploitation

Nœud de déploiement

Porte l'outil (Kolla-Ansible, OpenStack-Ansible, Juju), l'inventaire, les fichiers de configuration et les secrets. Atteint tous les nœuds en SSH par le réseau de gestion. **À sauvegarder** : sans lui, plus de reconfiguration ni de montée de version.

Bastion

Point d'entrée unique des administrateurs vers le réseau de gestion : authentification forte, journal des sessions. Aucun autre chemin ne doit mener au SSH des nœuds.

Supervision et journaux

Prometheus et ses exporteurs, Grafana, collecte centralisée des journaux (Fluentd ou Vector vers OpenSearch). Placés hors des contrôleurs pour rester lisibles quand ceux-ci souffrent.

Base et bus dédiés

À grande échelle, MariaDB et RabbitMQ quittent les contrôleurs pour des machines dédiées, puis se multiplient par cellule Nova. Les API restent seules sur les contrôleurs.

Registre et dépôts locaux

Registre d'images de conteneurs et miroirs de paquets : obligatoires en environnement déconnecté d'Internet, utiles partout pour des déploiements reproductibles.

Services d'infrastructure

NTP, DNS, éventuellement PXE et DHCP d'installation (MAAS, Ironic, Bifrost). Tous les nœuds en dépendent dès le démarrage.

> Aucun de ces nœuds n'est sur le chemin d'une requête d'utilisateur : leur panne ne coupe ni les API ni les instances. Elle prive en revanche l'équipe de ses outils — au pire moment si elle coïncide avec un incident.

Répartition des services par type de nœud

Processus	Contrôle	Calcul	Passerelle	Stockage	Remarque
HAProxy, Keepalived	●	—	—	—	Portent les deux adresses virtuelles
MariaDB, RabbitMQ, Memcached	●	—	—	—	Nœuds dédiés à grande échelle
keystone, glance-api, placement-api	●	—	—	—	REST et SQL uniquement, pas de bus
nova-api, scheduler, conductor, novncproxy	●	—	—	—	
nova-compute, libvirt	—	●	—	—	Un nova-compute par hyperviseur
neutron-server, ovn-northd, bases OVN	●	—	—	—	
ovn-controller, Open vSwitch	—	●	●	—	Sur tout nœud qui commute du trafic d'instances
ovn-metadata-agent	—	●	—	—	
cinder-api, cinder-scheduler	●	—	—	—	
cinder-volume, cinder-backup	●	—	—	○	Sur le nœud de stockage avec LVM
Ceph MON, MGR	○	—	—	●	Sur les contrôleurs dans un petit cloud
Ceph OSD	—	○	—	●	Sur le calcul en hyperconvergé
heat-api, heat-engine, horizon	●	—	—	—	

● emplacement habituel · ○ variante courante. Vérification sur un déploiement réel : `openstack compute service list` , `openstack volume service list` , `openstack network agent list` .

Trois topologies de référence

A Tout-en-un

Lab, démonstration, développement

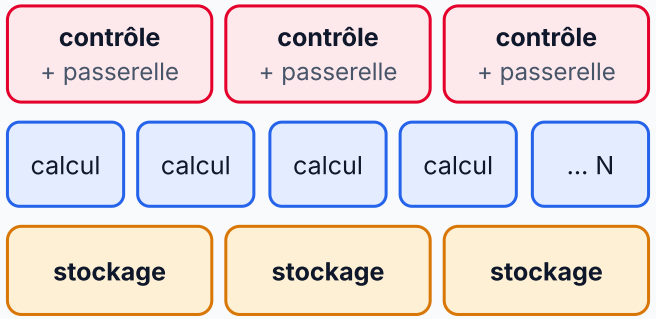
1 machine

contrôle + calcul
+ passerelle + stockage

- DevStack, Kolla all-in-one, Sunbeam mono-nœud
- Aucune haute disponibilité
- Les mêmes processus et les mêmes API qu'en production

B Standard : 3 + N

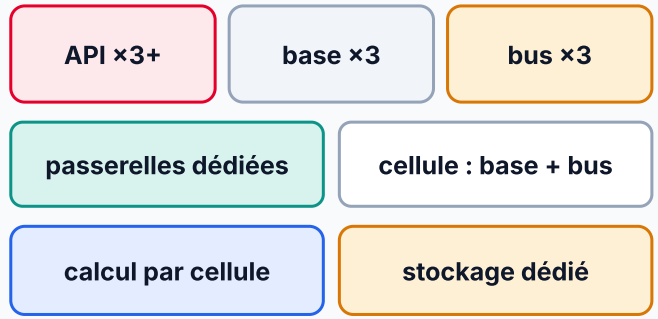
Production courante



- Galera, RabbitMQ, OVN : quorum à trois
- Calcul et stockage croissent séparément
- Convient jusqu'à quelques centaines d'hyperviseurs

C Grande échelle

Plusieurs centaines d'hyperviseurs et au-delà



- Chaque rôle sur ses propres machines
- Cellules Nova : une base et un bus par groupe d'hyperviseurs
- Régions multiples si plusieurs sites

Variante hyperconvergée : chaque nœud de calcul porte aussi des OSD Ceph. Moins de machines, mais calcul et stockage se disputent CPU, mémoire et réseau, et une panne de nœud touche les deux à la fois.

Que se passe-t-il quand un nœud tombe ?

Nœud perdu	Effet immédiat	Ce qui continue de fonctionner	Reprise
1 contrôleur sur 3	Les VIP basculent en quelques secondes ; quelques requêtes en cours échouent	Toutes les API ; Galera, RabbitMQ et OVN gardent leur quorum	Automatique ; réparer puis réintégrer le nœud
2 contrôleurs sur 3	Perte de quorum : base et bus se mettent en retrait, les API ne répondent plus	Les instances, leurs réseaux et leurs volumes	Manuelle : rétablir le quorum (amorçage Galera)
1 nœud de calcul	Ses instances s'arrêtent ; le service passe « down » après 60 s	Tout le reste du cloud	Évacuation vers un autre hôte (manuelle ou Masakari) si le disque est partagé ou sur volume
1 passerelle sur 2	Brève coupure du trafic nord-sud des routeurs qu'elle portait	Trafic entre instances (est-ouest), autres routeurs	Automatique : BFD, bascule vers le chassis suivant
1 nœud OSD Ceph	Cluster « dégradé », puis reconstruction des réplicas	Toutes les E/S, avec une latence accrue	Automatique tant que les réplicas restants suffisent
Nœud de déploiement	Aucun effet sur le cloud en service	Tout	Restaurer inventaire, configuration et secrets

Règle à retenir : la perte du plan de contrôle fige le cloud dans son état ; elle ne l'éteint pas. À l'inverse, la perte d'un nœud du plan de données touche immédiatement des charges utilisateur — d'où l'intérêt de répartir les instances par zones de disponibilité et groupes d'anti-affinité.

Dimensionner chaque rôle : la ressource qui limite

Rôle	Ressource critique	Ce qui la consomme	Règle pratique
Contrôle	Mémoire et latence disque	Workers des API (un par cœur par défaut), MariaDB, RabbitMQ	Disques SSD ou NVMe pour la base ; borner le nombre de workers par service
Calcul	Cœurs et mémoire vendables	Les instances, plus la part réservée à l'hôte	Fixer `cpu_allocation_ratio`, `ram_allocation_ratio` et `reserved_host_memory_mb`
Passerelle	Débit réseau et CPU de commutation	NAT et routage du trafic nord-sud	Interfaces 25 GbE ou plus, DPDK si le débit l'exige
Stockage Ceph	Disques, mémoire par OSD, réseau	Réplication ×3, reconstructions après panne	Prévoir de la mémoire pour chaque OSD ; réseau cluster dédié et rapide
Base et bus dédiés	Latence disque, nombre de connexions	Chaque processus de service ouvre ses propres connexions	Surveiller `max_connections` et la profondeur des files

Trois questions à se poser pour un nœud de calcul

- Quel taux de surallocation CPU la charge tolère-t-elle ? Aucun pour du temps réel ou du NFV (CPU dédiés).
- Combien de mémoire garder pour l'hôte : noyau, OVS, OSD éventuels ?
- Le disque éphémère est-il local ou sur Ceph ? Cela décide de la migration à chaud.

Côté contrôle, la charge suit le nombre d'hyperviseurs, pas le nombre d'instances : chaque nova-compute envoie ses rapports périodiques, chaque ovn-controller garde une connexion ouverte vers la base Southbound.

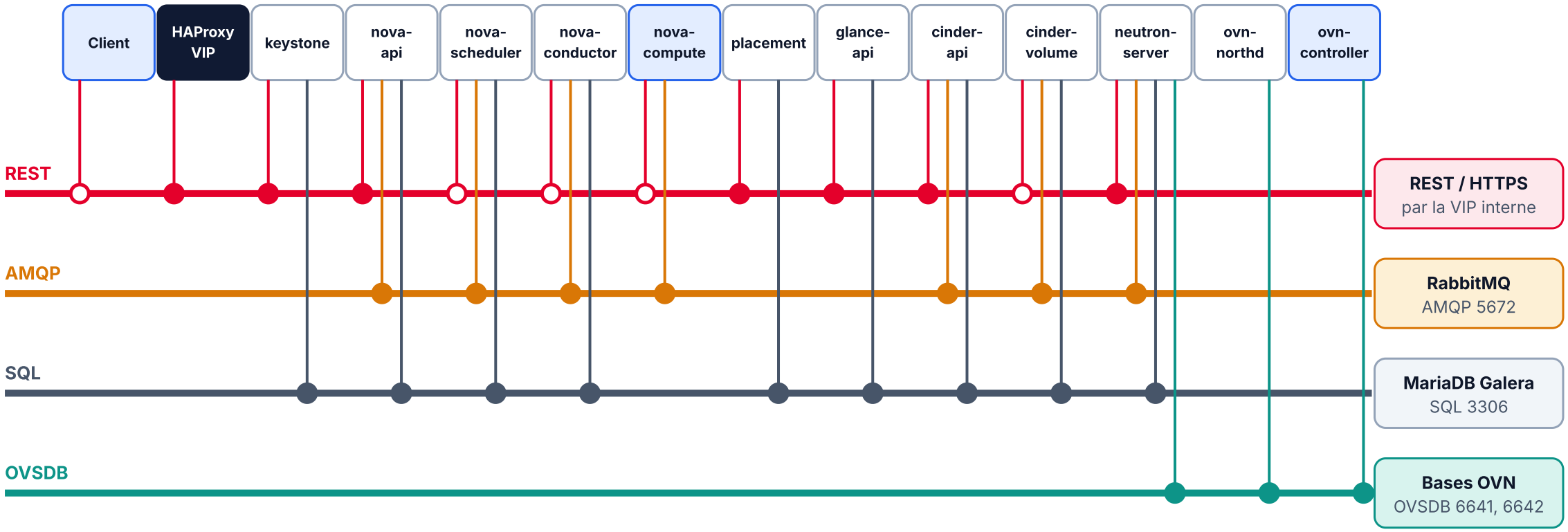
02

Communication entre composants

Quatre protocoles se partagent le plan de contrôle. Savoir lequel relie deux processus, c'est savoir où chercher quand une opération reste bloquée.

- La carte des protocoles : REST, AMQP, SQL, OVSDB
- AMQP et oslo.messaging : call, cast, fanout
- OVN, puis Nova ↔ Neutron et Nova ↔ Cinder
- Anatomie d'une requête API, points d'accès, jetons
- Conductor, base de données, RabbitMQ en cluster
- Heartbeats, identifiant de requête, diagnostic

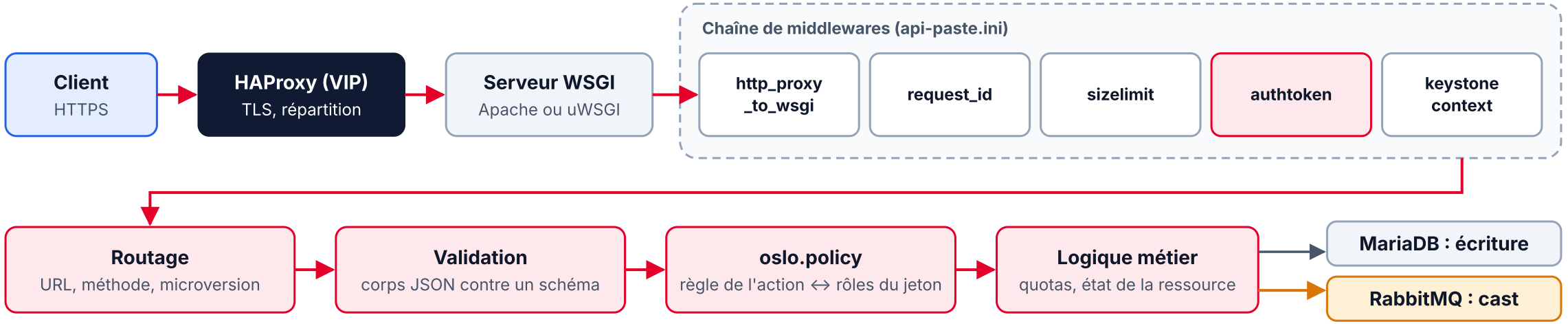
La carte des protocoles : qui parle quoi



nova-compute : REST et AMQP seulement — jamais de SQL, tout passe par nova-conductor.
keystone, placement, glance-api : REST et SQL seulement — aucun bus de messages.
ovn-northd, ovn-controller : OVSDB seulement — neutron-server fait le lien entre les deux mondes.

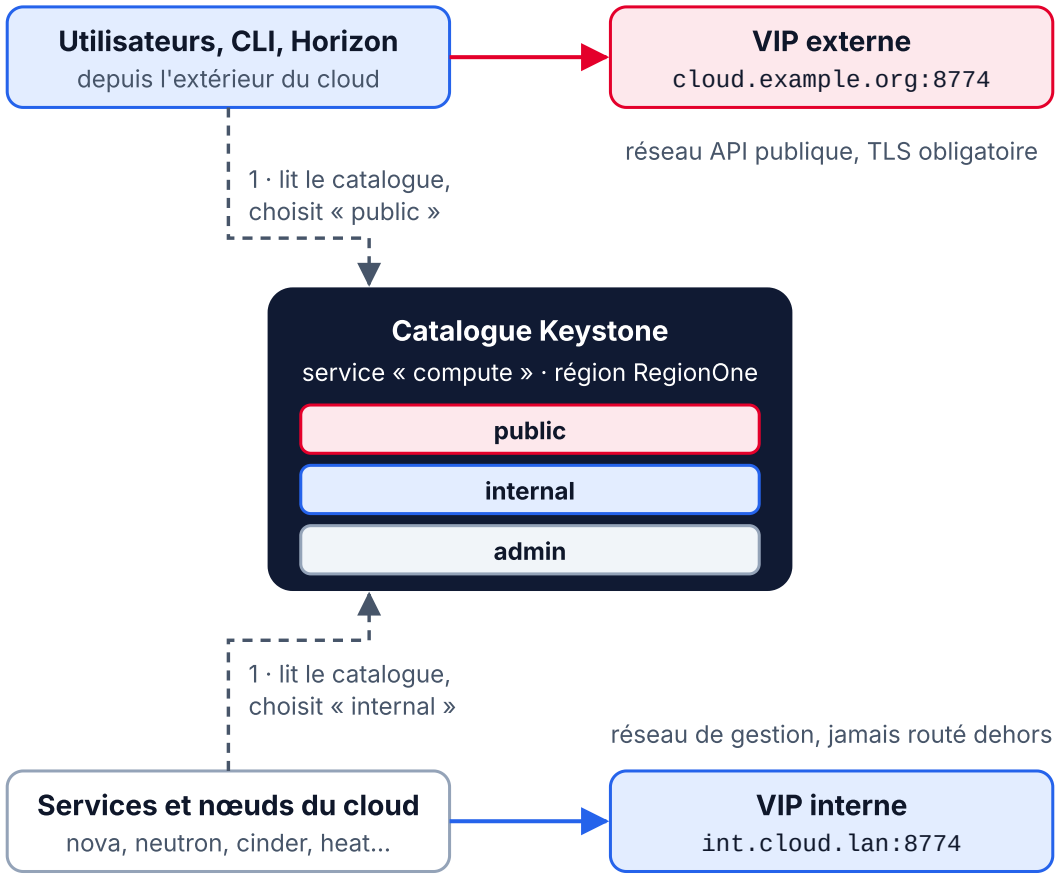
 Codes d'erreur : module 3

Anatomie d'une requête API



Étage	Ce qu'il fait	Ce qu'il transmet à l'étage suivant
HAProxy	Termine TLS, choisit un contrôleur sain	En-têtes `X-Forwarded-For`, `X-Forwarded-Proto`
request_id	Crée l'identifiant de la requête	`req-…`, renvoyé au client dans `X-OpenStack-Request-ID`
sizelimit	Refuse un corps de requête trop volumineux	La requête, inchangée
authtoken	Valide le jeton auprès de Keystone (résultat mis en cache dans Memcached)	Identité : `X-User-Id`, `X-Project-Id`, `X-Roles`
keystonecontext	Construit le contexte de la requête	Identité et identifiant, propagés ensuite dans chaque message RPC
Routage, validation	Associe l'URL à une action, vérifie la microversion et le corps JSON	Action et paramètres validés
oslo.policy	Compare la règle de l'action aux rôles et au projet du jeton	Autorisation accordée
Logique métier	Vérifie quotas et état, écrit en base, émet le RPC	Réponse **202** au client, sans attendre la fin

Points d'accès : public, interne, admin




```
$ openstack endpoint list --service compute -c Interface -c URL
+-----------+-----------------------------------------+
| Interface | URL                                     |
+-----------+-----------------------------------------+
| public    | https://cloud.example.org:8774/v2.1     |
| internal  | https://int.cloud.lan:8774/v2.1         |
| admin     | https://int.cloud.lan:8774/v2.1         |
+-----------+-----------------------------------------+
```

```
# nova.conf — Nova appelle les autres services en interne
[neutron]
valid_interfaces = internal
region_name = RegionOne
[glance]
valid_interfaces = internal
[placement]
valid_interfaces = internal
[cinder]
catalog_info = volumev3::internalURL
```

- Un service ne connaît **aucune adresse en dur** : il la demande au catalogue, à chaque nouveau jeton.
- L'interface **admin** n'a plus de rôle propre : elle pointe en général sur la même URL que l'interne.

Les ports à connaître

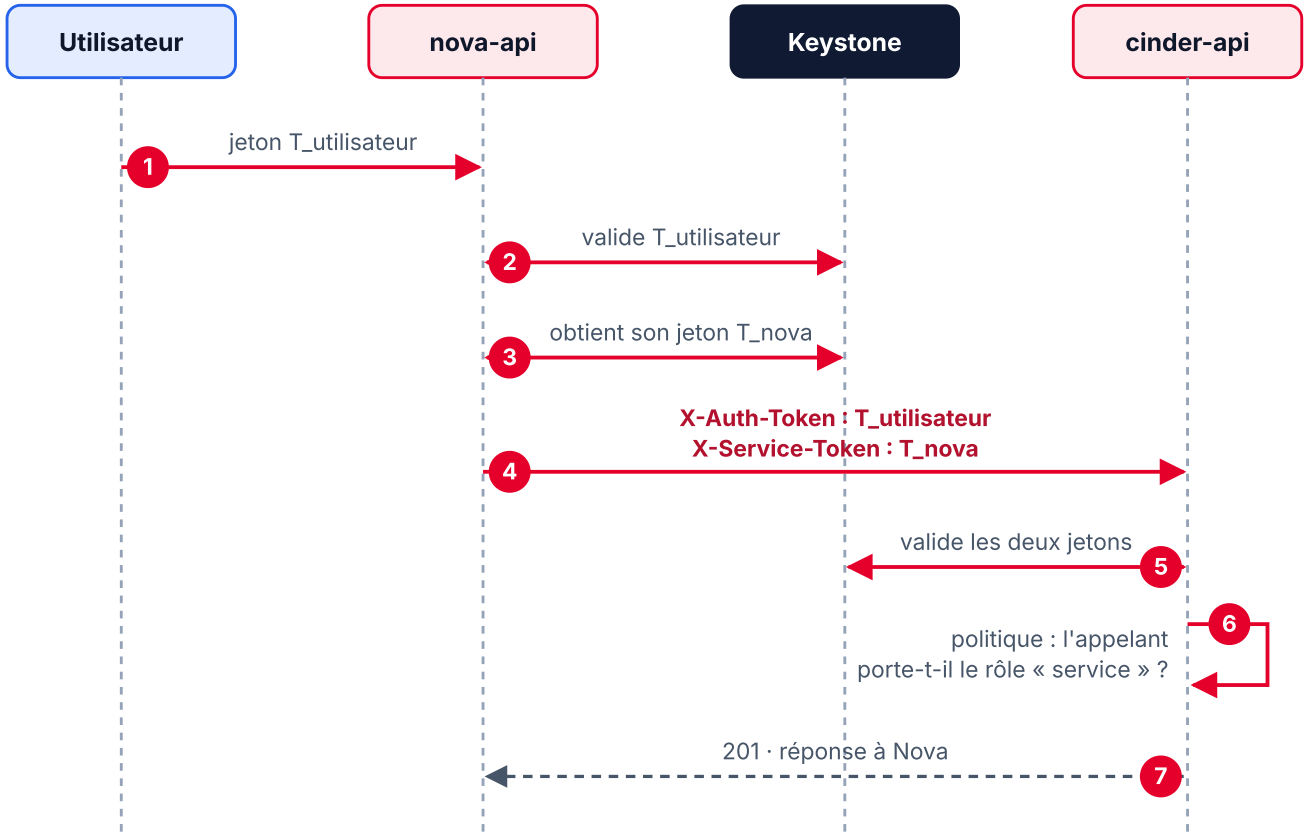
API principales	Port
Keystone	5000
Nova	8774
Nova, métadonnées	8775
Placement	8778
Neutron	9696
Cinder	8776
Glance	9292
Swift (proxy)	8080
Horizon	80, 443
Console noVNC	6080

API complémentaires	Port
Heat	8004
Octavia	9876
Barbican	9311
Designate	9001
Manila	8786
Magnum	9511
Ironic	6385

> Ce sont les valeurs par défaut. Derrière HAProxy, un déploiement peut tout publier sur 443 avec un chemin ou un nom par service : le catalogue fait foi.

Infrastructure et données	Port
MariaDB	3306
Galera	4567, 4568, 4444
RabbitMQ	5672 (TLS 5671)
RabbitMQ, cluster	4369, 25672
Memcached	11211
OVN NB, SB	6641, 6642
OVN, Raft	6643, 6644
Geneve, VXLAN	UDP 6081, 4789
libvirt	16509 (TLS 16514)
Migration QEMU	49152–49215
Consoles VNC	5900–5999
Ceph MON	3300, 6789
Ceph OSD	6800–7300
iSCSI (LVM)	3260

Authentification entre services : jeton d'utilisateur et jeton de service



- Le jeton de l'utilisateur dit **pour qui** l'action est faite (utilisateur, projet, rôles).
- Le jeton de service dit **qui relaie** la demande : un compte technique portant le rôle `service`.
- Si le jeton de l'utilisateur expire pendant une opération longue (migration, instantané), l'appel reste accepté grâce au jeton de service.
- Certaines actions de Cinder sont réservées aux appels portant ce rôle : sans jeton de service, l'attachement de volumes échoue.

```
# nova.conf — côté appelant
[service_user]
send_service_user_token = true
auth_type = password
username = nova
project_name = service

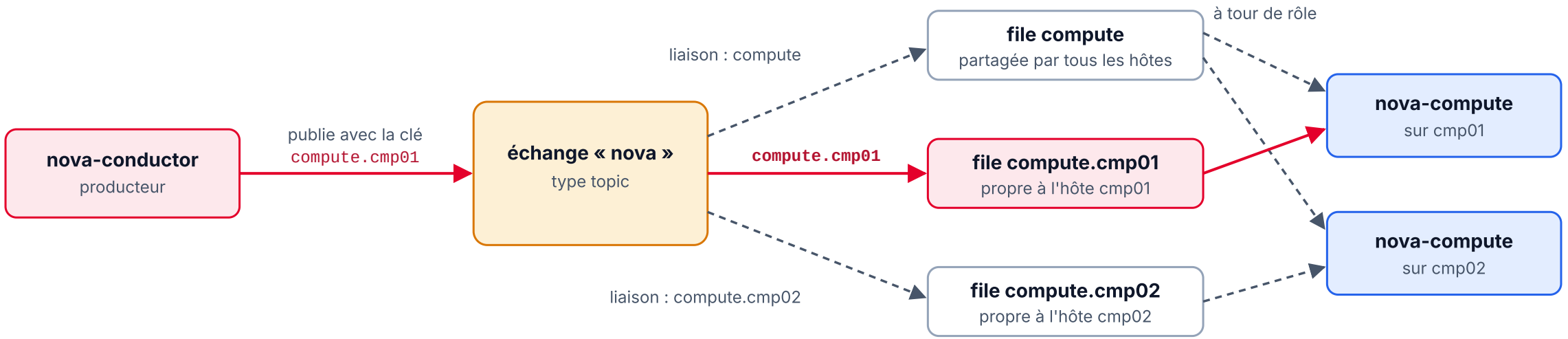
# cinder.conf — côté appelé
[keystone_authtoken]
service_token_roles = service
service_token_roles_required = true
```

Qui appelle qui en REST

Appelant	Appelé	Pour faire quoi	Appel caractéristique
Tous les services	Keystone	Valider les jetons reçus, obtenir leur propre jeton, lire le catalogue	`GET /v3/auth/tokens`
Nova	Placement	Chercher les hôtes candidats, réserver puis libérer les ressources, publier les inventaires	`GET /allocation_candidates`
Nova	Glance	Lire les propriétés d'une image, la télécharger, téléverser un instantané	`GET /v2/images/{id}/file`
Nova	Neutron	Créer ou lier les ports, lire réseaux et groupes de sécurité	`PUT /v2.0/ports/{id}`
Nova	Cinder	Réserver, connecter et détacher les volumes	`POST /v3/attachments`
Neutron	Nova	Signaler qu'un port est prêt ou qu'un réseau a changé	`POST /os-server-external-events`
Neutron	Placement	Déclarer la bande passante garantie (QoS) et les segments de réseaux routés	`PUT /resource_providers/…`
Cinder	Glance	Créer un volume à partir d'une image, exporter un volume en image	`GET /v2/images/{id}`
Cinder	Nova	Étendre un volume attaché, coordonner certains instantanés	`POST /os-server-external-events`
Cinder, Nova, Octavia	Barbican	Récupérer clés de chiffrement et certificats	`GET /v1/secrets/{id}`
Heat, Magnum, Octavia	Nova, Neutron, Cinder…	Créer les ressources décrites, comme le ferait un utilisateur	API publiques des services

Constantes : l'appelant découvre l'URL dans le catalogue (interface `internal`), présente un jeton, et ne touche jamais à la base de l'autre service. Une panne de Keystone ou de la VIP interne paralyse donc tous ces échanges d'un coup.

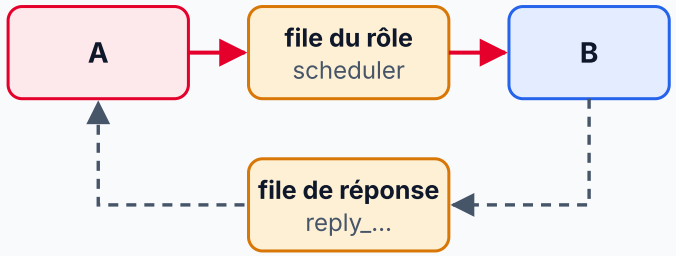
AMQP en cinq notions



Notion	Définition	Dans OpenStack
Échange (exchange)	Point d'entrée du courtier : reçoit les messages et les aiguille, sans les conserver	Un échange de type *topic* par service (`nova` , `cinder` , `neutron` ...), des échanges *fanout* et *direct*
File (queue)	Tampon où les messages attendent un consommateur	Une file par rôle, une par hôte, une par diffusion, une par réponse
Liaison (binding)	Règle reliant un échange à une file selon la clé de routage	Créée par le service lui-même à son démarrage
Clé de routage	Étiquette posée sur le message par l'émetteur	Le nom du rôle (`scheduler`) ou rôle + hôte (`compute.cmp01`)
Consommateur	Processus abonné à une file ; il acquitte chaque message traité	Plusieurs consommateurs sur une même file = répartition de charge

oslo.messaging : call, cast, fanout

call — appel avec réponse



A attend la réponse, au plus rpc_response_timeout (60 s par défaut).

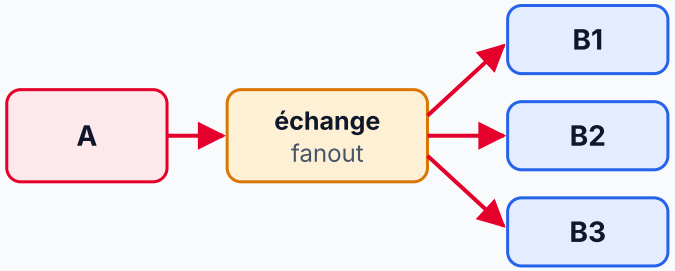
cast — ordre sans réponse



A poursuit aussitôt. Le résultat se lira plus tard dans l'état de la ressource (BUILD → ACTIVE ou ERROR).

C'est le mode des opérations longues : création, migration, attachement.

fanout — diffusion à tous

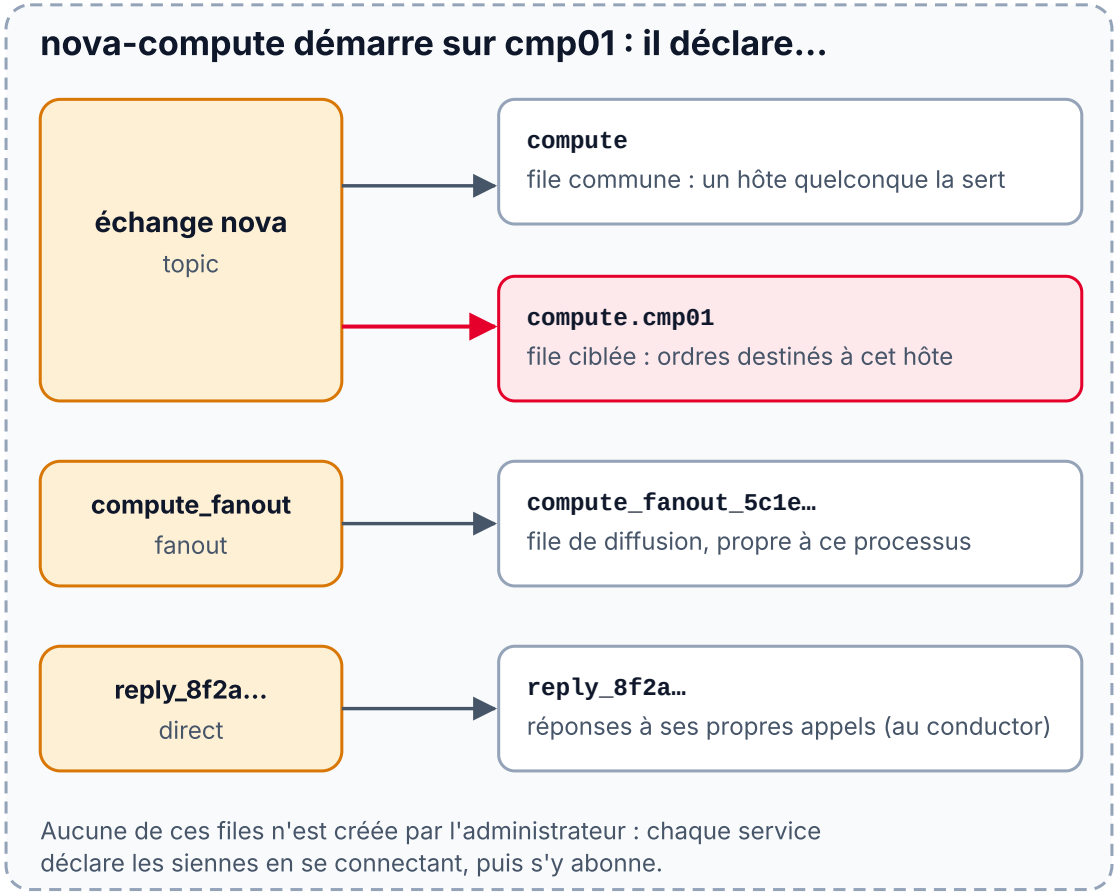


Chaque abonné reçoit sa propre copie, dans une file qui lui est réservée.

Mode	Bloquant ?	Destinataires	En cas de destinataire absent	Exemple
call	Oui	Un seul, pris à tour de rôle ou ciblé par hôte	Exception `MessagingTimeout` chez l'appelant	conductor → scheduler : `select_destinations`
cast	Non	Un seul	Le message attend dans la file ; l'opération paraît figée	conductor → compute : `build_and_run_instance`
fanout	Non	Tous les abonnés du moment	Les absents ne recevront jamais le message	cinder-volume → tous les cinder-scheduler : capacités du backend

Lecture des colonnes : module 3

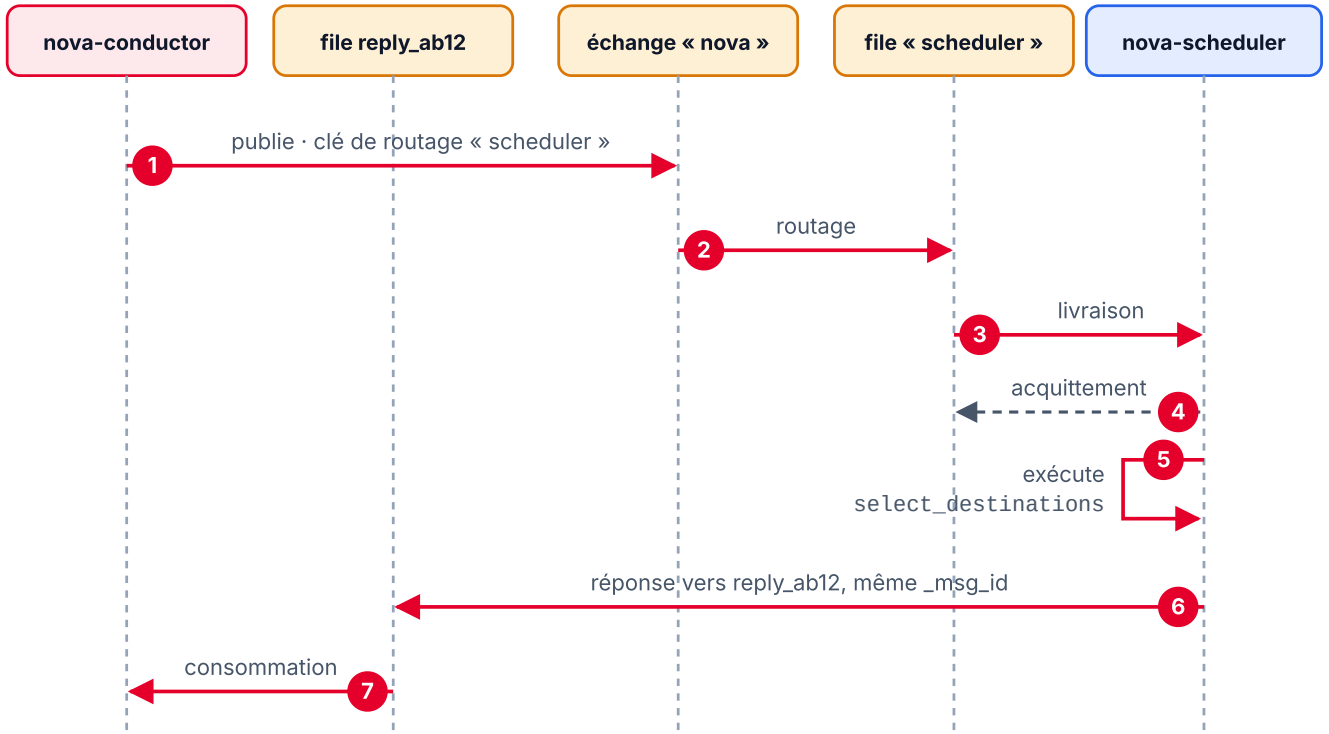
Les files créées par un service, vues depuis RabbitMQ




```
$ rabbitmqctl list_queues name messages consumers
compute                                0     12
compute.cmp01                          0      1
compute.cmp02                          0      1
compute_fanout_5c1e…                   0      1
conductor                              0      3
scheduler                              0      3
cinder-scheduler                       0      3
cinder-volume.ctl01@ceph               0      1
reply_8f2a…                            0      1
versioned_notifications.info        4812      0
```

- Chaque ligne est une file déclarée par un service : par rôle (`conductor`), par hôte (`compute.cmp01`), de diffusion ou de réponse.
- Dernière ligne : des **notifications** émises sans que personne ne les lise. La file grossit indéfiniment — piège classique, traité plus loin.

Séquence d'un appel RPC « call »



Sans réponse sous rpc_response_timeout : exception MessagingTimeout chez l'appelant.
Trois nova-scheduler abonnés à la même file : RabbitMQ les sert à tour de rôle.

```
# Message publié à l'étape 1 (simplifié)
{
  "method": "select_destinations",
  "args": { "spec_obj": {…}, … },
  "version": "4.5",
  "_msg_id": "c0a1f3…",
  "_reply_q": "reply_ab12…",
  "_context_request_id": "req-5f1c2b7e…",
  "_context_project_id": "c3d4…",
  "_context_user_id": "a1b2…"
}
```

- `method` et `args` : la fonction distante à exécuter.
- `version` : version de l'interface RPC ; elle permet à des services de versions différentes de cohabiter pendant une montée de version.
- `_msg_id` et `_reply_q` : de quoi renvoyer la réponse au bon appelant.
- `_context_*` : identité et identifiant de requête, transportés d'un processus à l'autre.

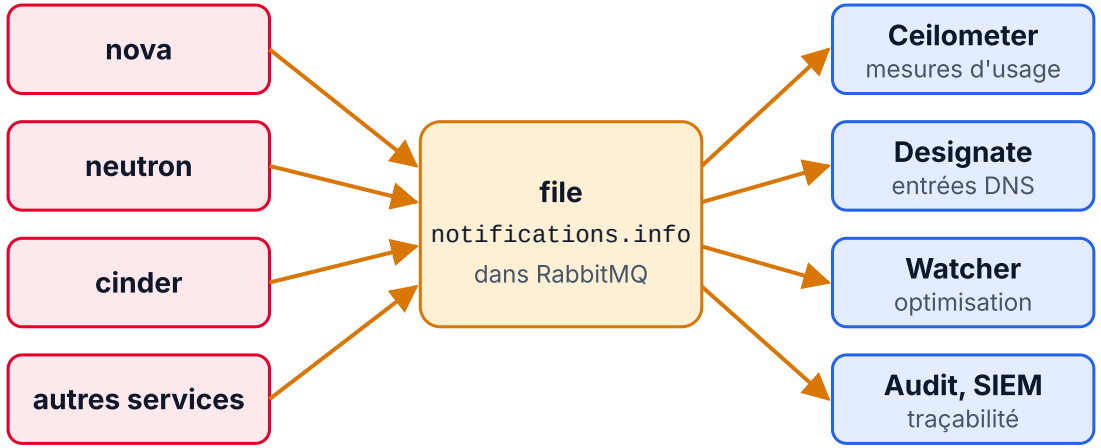
Les files à reconnaître, service par service

Service	Processus abonné	File (topic)	Ce qui y arrive
Nova	nova-scheduler	`scheduler`	Demandes de placement émises par les conductors
Nova	nova-conductor	`conductor`	Tâches longues venues de l'API, accès base demandés par les computes
Nova	nova-compute	`compute`, `compute.<hôte>`	Création, arrêt, migration, attachement, événements externes
Cinder	cinder-scheduler	`cinder-scheduler`	Demandes de création à placer ; capacités des backends (fanout)
Cinder	cinder-volume	`cinder-volume.<hôte>@<backend>`	Création, extension, instantané, connexion d'un volume
Neutron (ML2/OVS)	neutron-server	`q-plugin`, `q-reports-plugin`	Demandes et rapports d'état des agents
Neutron (ML2/OVS)	agents L2, L3, DHCP	`q-agent-notifier-…`, `l3_agent.<hôte>`, `dhcp_agent.<hôte>`	Mises à jour de ports, routeurs, réseaux
Heat	heat-engine	`engine`	Opérations sur les piles reçues par heat-api
Tous	l'appelant lui-même	`reply_<uuid>`	Réponses à ses appels « call »
Tous (si activé)	Ceilometer et autres	`notifications.info`, `versioned_notifications.info`	Événements métier, à sens unique

Aucun bus de messages pour Keystone, Glance, Placement et Swift : ces services ne font que du REST et du SQL (ou du stockage). Inutile d'y chercher un problème RabbitMQ.

Avec ML2/OVN, neutron-server ne pilote plus d'agents par RPC : il écrit dans la base Northbound. Les files `q-*` disparaissent presque toutes.

Les notifications : un second flux, à sens unique



Exemple d'événement émis par Nova

```
event_type : compute.instance.create.end
publisher_id : compute.cmp01
payload : instance_id, tenant_id, flavor, host, state…
```

Chaque étape d'une opération produit un événement « .start » puis « .end » (ou « .error »). L'émetteur n'attend rien en retour et ignore qui écoute.

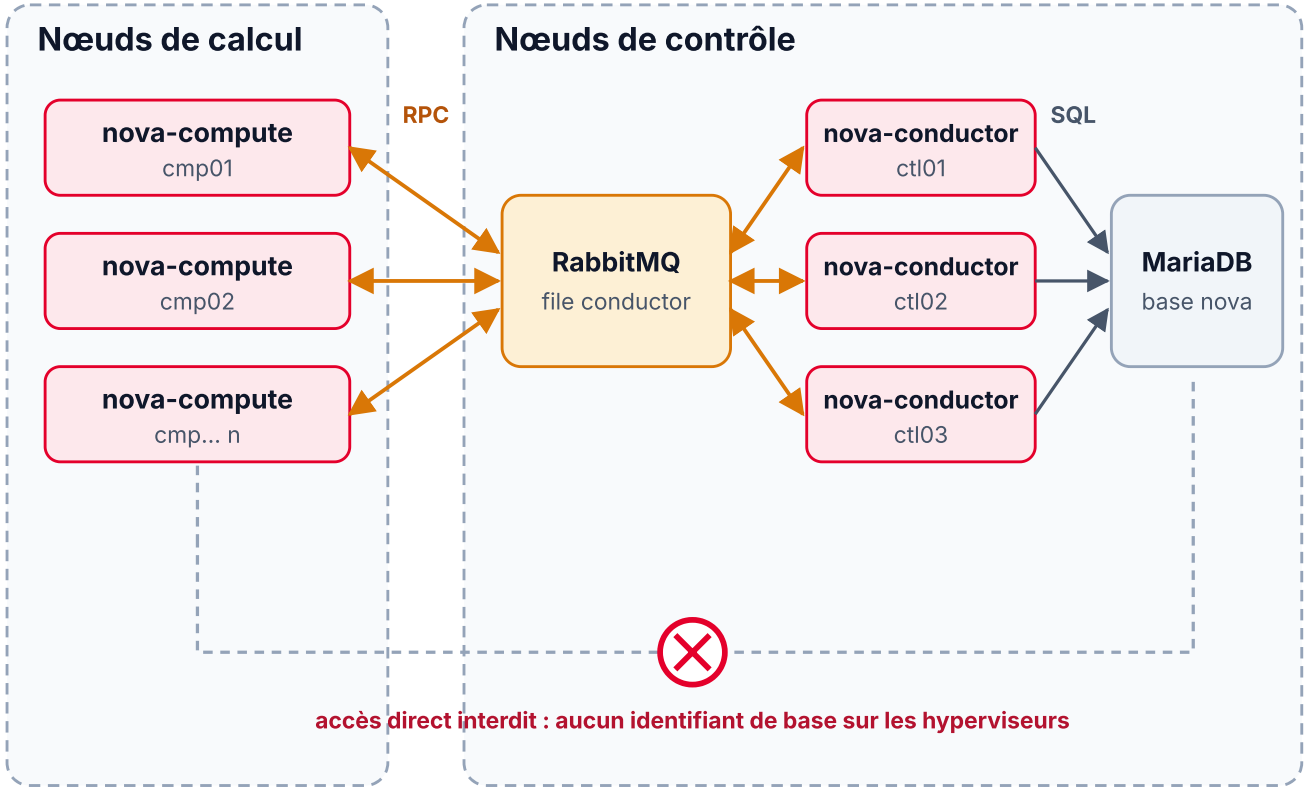
	RPC	Notification
But	Faire exécuter une action	Informer qu'une action a eu lieu
Destinataire	Un processus du même service	Tout consommateur externe
Si personne n'écoute	L'opération échoue ou attend	Les messages s'empilent

```
[oslo_messaging_notifications]
driver = messagingv2   # noop pour ne rien émettre
topics = notifications
# bus distinct possible, pour isoler la charge :
transport_url = rabbit://notif:***@mq-notif:5672//

[notifications]         # nova.conf
notification_format = unversioned
```

Piège : activer les notifications sans consommateur remplit RabbitMQ jusqu'à saturer sa mémoire, et c'est alors le RPC qui s'arrête. Sans Ceilometer ni équivalent : `driver = noop`.

Le motif « conductor » : aucun hyperviseur ne touche à la base



Quatre raisons d'être

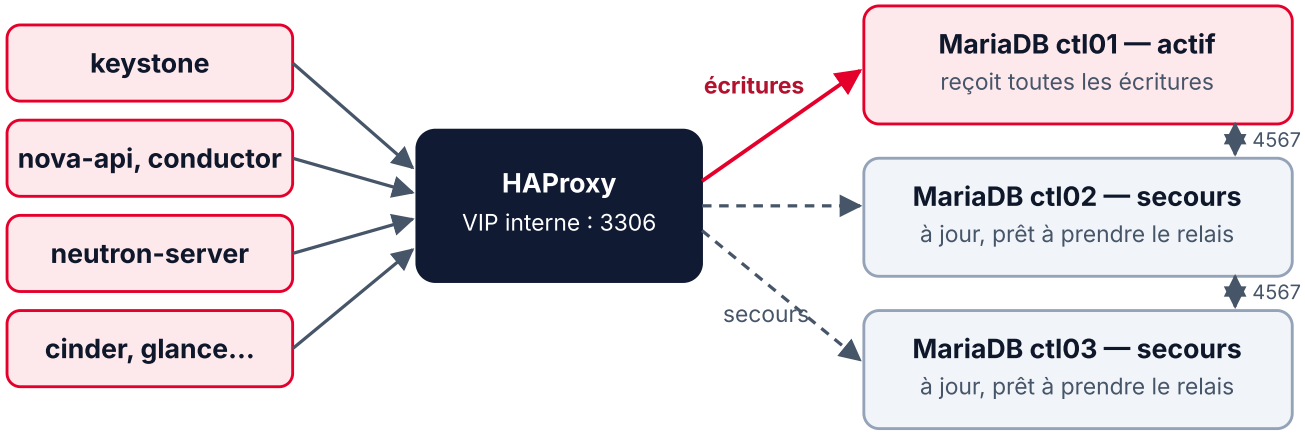
- **Sécurité :** un hyperviseur exécute du code de locataires ; s'il est compromis, il ne détient aucun mot de passe de base de données.
- **Montées de version :** le conductor traduit les objets entre versions ; des nova-compute anciens cohabitent avec un contrôle déjà mis à jour.
- **Mise à l'échelle :** quelques conductors mutualisent les connexions SQL de milliers d'hyperviseurs.
- **Orchestration :** il enchaîne les étapes des opérations longues (création, redimensionnement, migration) entre scheduler et computes.

Avec les cellules : un « super-conductor » au niveau de l'API répartit vers le conductor de chaque cellule, qui possède sa base et son bus.

Cinder n'applique pas ce motif : `cinder-volume` accède directement à sa base. Il tourne donc sur des nœuds de confiance.

 Galera en détail : module 3

La base de données : une par service, un seul nœud en écriture



Un schéma par service, un compte par service

`keystone`	`nova_api`	`nova_cell0`	`nova`	`placement`
`neutron`	`cinder`	`glance`	`heat`	`octavia…`

Réplication Galera synchrone entre les trois nœuds ; HAProxy dirige toutes les écritures vers un seul.

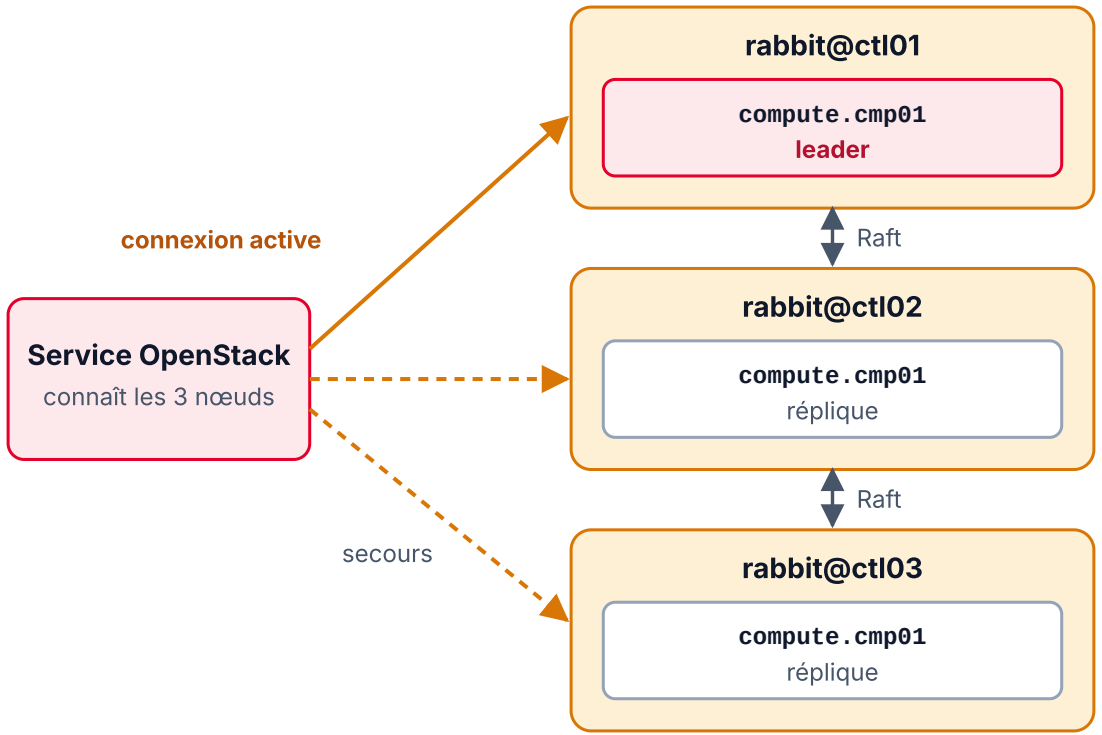
Chaque service ne connaît que la VIP interne et son propre compte : il ignore quel nœud est actif et ne voit jamais les schémas des autres services.

```
[database]            # nova.conf
connection = mysql+pymysql://nova:***@vip-int/nova
max_pool_size = 5
max_overflow = 50
connection_recycle_time = 600

[api_database]
connection = mysql+pymysql://nova:***@vip-int/nova_api
```

- **oslo.db** (SQLAlchemy) gère le pool de connexions et rejoue une requête après un « deadlock ».
- Chaque processus et chaque worker ouvre son propre pool : le total doit rester sous `max_connections`.
- Les schémas évoluent à chaque montée de version : `nova-manage db sync`, `neutron-db-manage upgrade`…

RabbitMQ en cluster : files à quorum et connexions directes



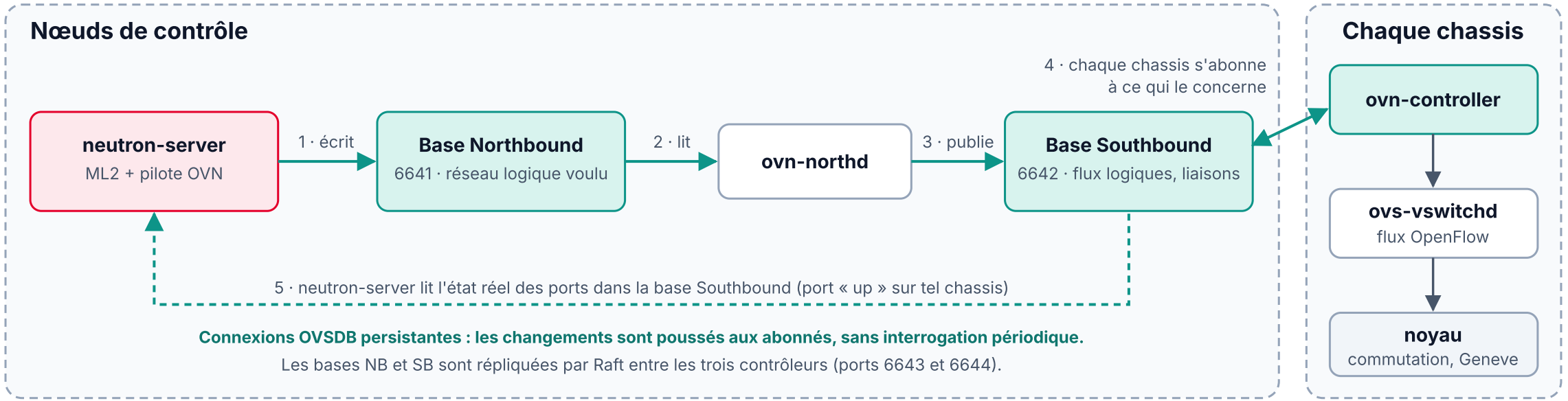
Un message publié n'est confirmé qu'une fois écrit sur la majorité des nœuds.
Si le leader disparaît, une réplique est élue : la file survit sans perte.

```
[DEFAULT]
transport_url = rabbit://openstack:***@10.20.0.11:5672,
    openstack:***@10.20.0.12:5672,
    openstack:***@10.20.0.13:5672//

[oslo_messaging_rabbit]
rabbit_quorum_queue = true
heartbeat_timeout_threshold = 60
ssl = true
```

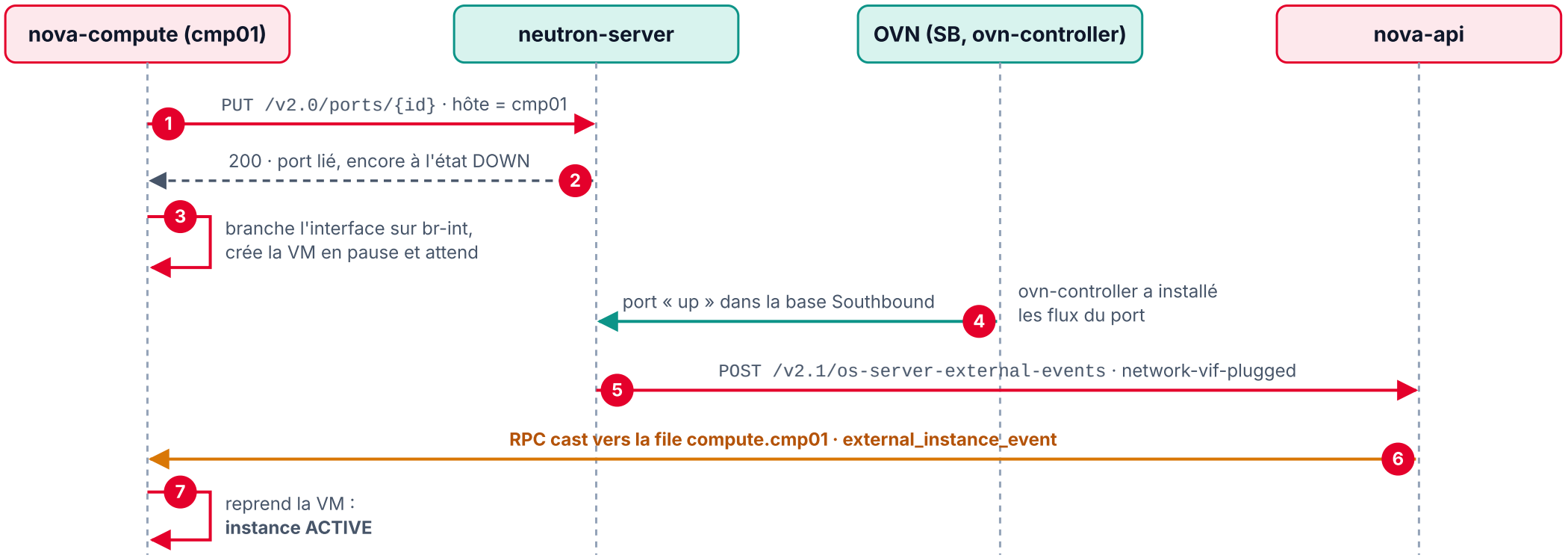
- Les services se connectent **directement** aux nœuds RabbitMQ, sans passer par HAProxy : la bibliothèque gère elle-même la reconnexion.
- **Battements AMQP :** une connexion muette au-delà du seuil est fermée puis rétablie ; un pare-feu qui coupe les connexions inactives produit les mêmes symptômes.

Neutron avec OVN : OVSDB plutôt qu'AMQP



	ML2/OVN (référence)	ML2/Open vSwitch (historique)
Transport des ordres	OVSDB, de base en base ; chaque chassis s'abonne à la base Southbound	RPC AMQP : neutron-server ↔ agents, à travers RabbitMQ
Processus sur les hôtes	ovn-controller, ovn-metadata-agent	Agents OVS, L3, DHCP, métadonnées
Où vit l'état réseau	Base Neutron (référence) + bases NB / SB (traduction)	Base Neutron + état local de chaque agent
Dépendance à RabbitMQ	Faible : un bus en panne ne fige pas le réseau	Forte : sans bus, plus aucune mise à jour des agents
Point de vigilance	Cohérence entre base Neutron et base NB (outil de synchronisation)	Charge RPC qui croît avec le nombre d'agents et de ports

Nova ↔ Neutron : l'événement qui libère le démarrage

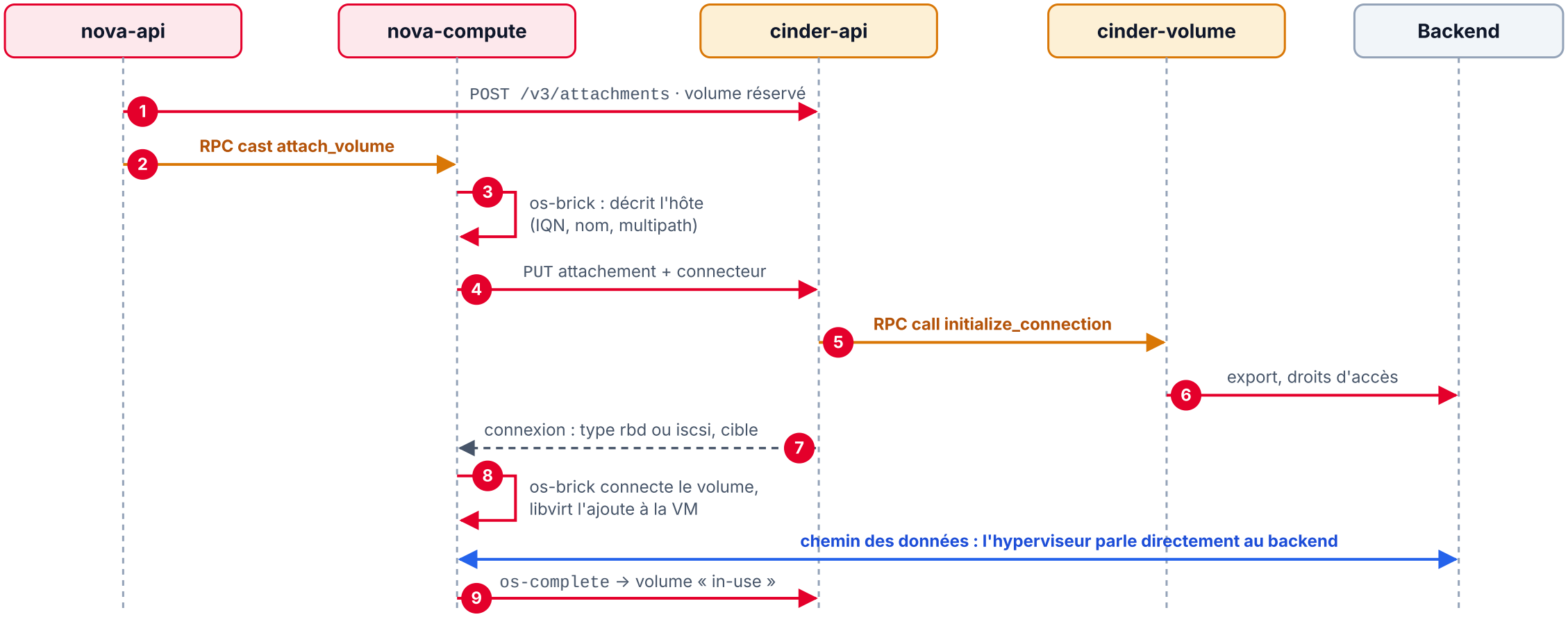


Trois protocoles en une seule opération : REST (1, 2, 5), OVSDB (4), RPC (6).

Si l'événement n'arrive pas dans le délai `vif_plugging_timeout` (300 s par défaut), l'instance passe en ERROR avec « Virtual Interface creation failed ». Causes habituelles : ovn-controller arrêté sur l'hôte, Neutron incapable de joindre l'API Nova.

Neutron a donc besoin d'un compte et d'un accès à Nova : section `[nova]` de `neutron.conf`, interface `internal`. C'est l'un des rares appels « à rebours » de l'architecture.

Nova ↔ Cinder : attacher un volume



Lecture : Nova ne sait rien du stockage. Il décrit l'hôte à Cinder (étape 4), Cinder répond comment s'y connecter (étape 7), puis s'efface. Entre services : REST ; à l'intérieur de chaque service : RPC ; pour les données : le protocole du backend, sans intermédiaire.

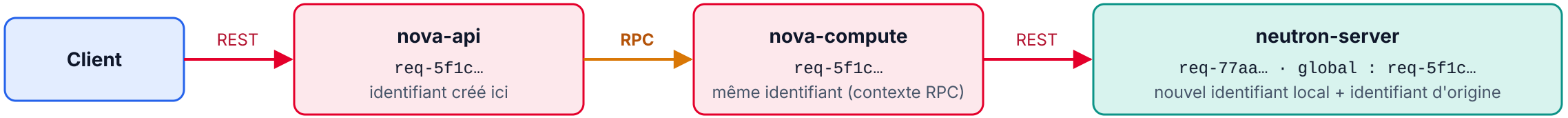
Battements de cœur : comment le cloud sait qu'un service est vivant

Composant	Mécanisme	Paramètres (valeurs par défaut)	Commande de contrôle
Services Nova	Chaque processus met à jour sa ligne dans la table `services` (les computes via le conductor)	`report_interval = 10` · `service_down_time = 60`	`openstack compute service list`
Services Cinder	Même principe, écriture directe en base	`report_interval = 10` · `service_down_time = 60`	`openstack volume service list`
Agents Neutron (ML2/OVS)	Rapport d'état envoyé par RPC à neutron-server	`report_interval = 30` · `agent_down_time = 75`	`openstack network agent list`
Chassis OVN	Chaque ovn-controller horodate sa ligne dans la base Southbound	—	`openstack network agent list`, `ovn-sbctl show`
Connexions RabbitMQ	Battements AMQP entre client et courtier	`heartbeat_timeout_threshold = 60`	`rabbitmqctl list_connections`
API derrière HAProxy	Contrôle de santé HTTP périodique de chaque backend	Selon la configuration HAProxy	Page de statistiques HAProxy
Galera	Appartenance au cluster, état de synchronisation	—	`SHOW STATUS LIKE 'wsrep_%'`

```
$ openstack compute service list
+----------------+-------+---------+-------+
| Binary         | Host  | Status  | State |
+----------------+-------+---------+-------+
| nova-scheduler | ctl01 | enabled | up    |
| nova-conductor | ctl01 | enabled | up    |
| nova-compute   | cmp01 | enabled | up    |
| nova-compute   | cmp02 | enabled | down  |
+----------------+-------+---------+-------+
```

- **State** (up / down) est **déduit** : dernier battement plus vieux que `service_down_time`.
- **Status** (enabled / disabled) est un **choix de l'administrateur**. Le scheduler ignore les hôtes « down » ou « disabled ».
- Un service « down » alors que son processus tourne : il n'arrive plus à écrire son battement — bus, base ou horloge.

Suivre une requête d'un service à l'autre



À l'intérieur d'un service, l'identifiant ne change pas. En changeant de service, il devient l'« identifiant global ».

```
# 1. Récupérer l'identifiant renvoyé au client
$ openstack --debug server create … 2>&1 | grep -i x-openstack-request-id
x-openstack-request-id: req-5f1c2b7e-9c41-4d0a-b7c2-3a1e6d0f8a11

# 2. nova-api.log, sur un contrôleur
INFO nova.osapi_compute.wsgi.server [req-5f1c2b7e-… - a1b2… c3d4…] "POST /v2.1/servers" status: 202

# 3. nova-compute.log, sur l'hyperviseur choisi — même identifiant
INFO nova.compute.manager [req-5f1c2b7e-… - a1b2… c3d4…] [instance: 9d0e…] Creating image(s)

# 4. neutron-server.log — identifiant local, puis identifiant global
INFO neutron.wsgi [req-77aa0c5d-… req-5f1c2b7e-… a1b2… c3d4…] "POST /v2.0/ports" status: 201
```

Méthode : centraliser les journaux de tous les nœuds, puis filtrer sur l'identifiant de requête. On reconstitue ainsi tout le parcours d'une opération, protocole après protocole, sans se connecter à chaque machine.

Du symptôme au protocole en cause

Symptôme	Protocole	Cause probable	Premier contrôle
503 sur une API	REST	HAProxy ne voit plus aucun backend sain, ou le service ne joint plus Keystone	Statistiques HAProxy, journal de l'API
401 par intermittence	REST	Horloges désynchronisées ; clés Fernet différentes d'un contrôleur à l'autre	`chronyc tracking`, dépôt de clés Keystone
`MessagingTimeout`	AMQP	Destinataire arrêté ou saturé ; file sans consommateur ; partition RabbitMQ	`rabbitmqctl list_queues`, liste des services
Instance figée en BUILD ou « scheduling »	AMQP	Cast resté dans une file : conductor, scheduler ou compute absent	`openstack compute service list`
Services « down » alors que les processus tournent	AMQP / SQL	Battements qui n'arrivent plus en base	RabbitMQ, Galera, horloge de l'hôte
`DBDeadlock`, `DBConnectionError`	SQL	Écritures sur plusieurs nœuds Galera ; quorum perdu ; trop de connexions	`wsrep_cluster_size`, `max_connections`
« Virtual Interface creation failed »	OVSDB + REST	ovn-controller arrêté sur l'hôte, ou Neutron ne joint pas l'API Nova	`ovn-sbctl show`, section `[nova]` de neutron.conf
Port qui reste DOWN	OVSDB	Chassis non enregistré, base Southbound injoignable depuis l'hôte	`ovs-vsctl get open . external_ids`
« No valid host was found »	—	Pas un problème de communication : aucun hôte ne satisfait la demande	Placement, filtres du scheduler, quotas

Réflexe : identifier d'abord **entre quels processus** l'opération s'est arrêtée (identifiant de requête dans les journaux), puis **quel protocole** les relie. La cause se trouve presque toujours dans l'un de quatre endroits : VIP et Keystone, RabbitMQ, Galera, bases OVN.

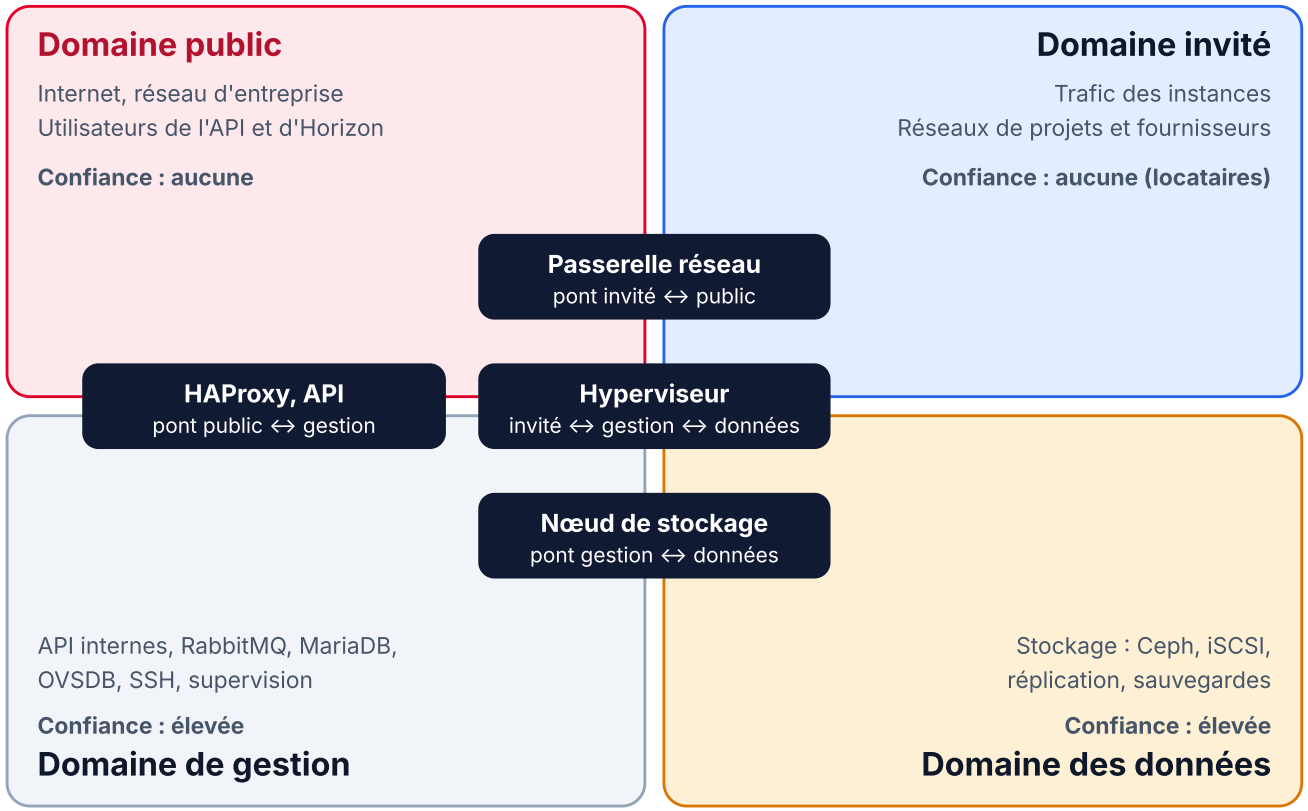
03

Segmentation réseau du plan de contrôle

Les flux décrits jusqu'ici ne circulent pas sur un réseau unique. Chaque famille de flux a le sien, pour des raisons de sécurité, de performance et de limitation des pannes.

- Les quatre domaines de sécurité
- Quel nœud sur quel réseau
- Câblage, matrice de flux, MTU
- Les réseaux d'un déploiement
- VIP interne, VIP externe, chiffrement
- Traduction en configuration

Pourquoi segmenter : quatre domaines de sécurité



Le guide de sécurité d'OpenStack distingue quatre **domaines**, de niveaux de confiance différents. Les composants à cheval sur deux domaines — les **ponts** — sont les points à durcir en priorité.

- **Sécurité :** une instance compromise ne doit jamais atteindre RabbitMQ, MariaDB ou le SSH des nœuds. Sur un réseau unique, rien ne l'en empêcherait.
- **Performance :** une reconstruction Ceph ou une migration à chaud sature un lien ; les battements de Galera et de RabbitMQ ne doivent pas en pâtir.
- **Limitation des pannes :** une tempête de diffusion sur un réseau de projet ne doit pas faire tomber le plan de contrôle.
- **Exploitation :** un réseau par usage rend les règles de pare-feu lisibles et les captures de trafic exploitables.

> Segmenter, c'est attribuer un **VLAN et un sous-réseau par famille de flux**, puis ne raccorder chaque nœud qu'aux réseaux dont son rôle a besoin.

3 · SEGMENTATION RÉSEAU

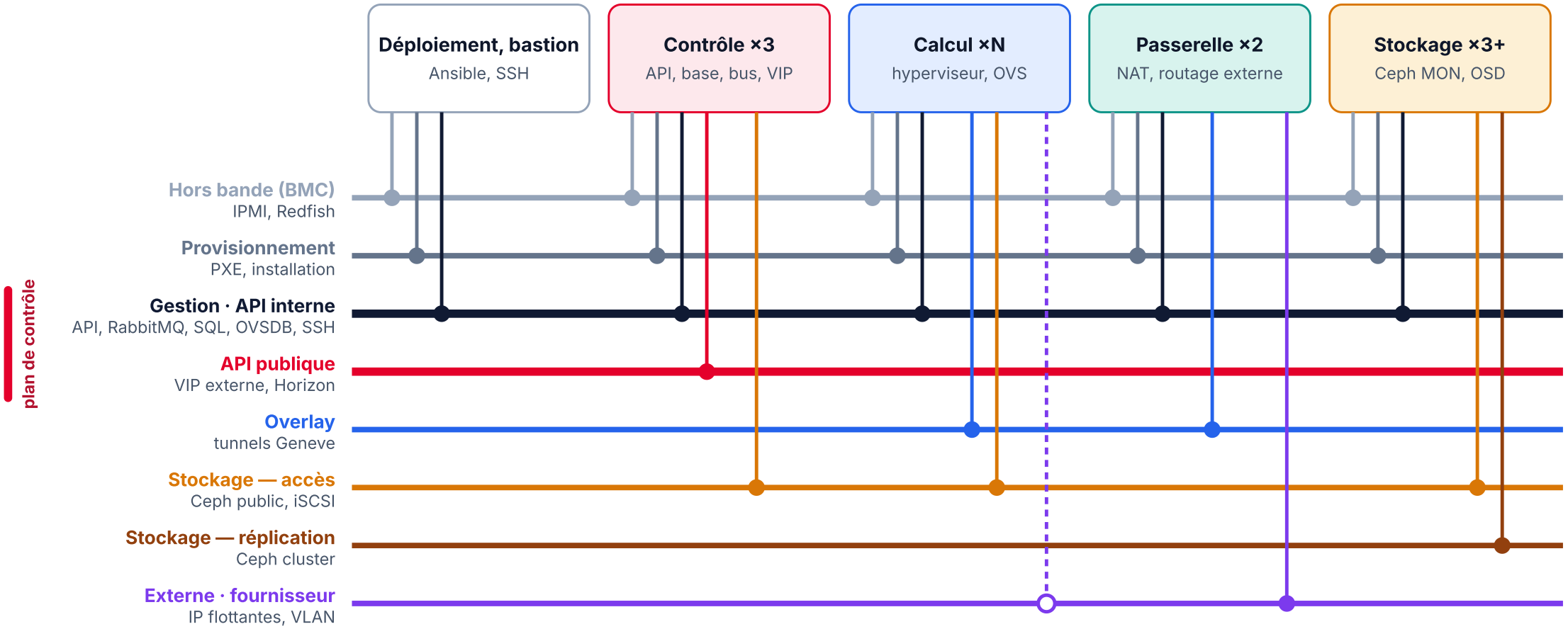
Les réseaux d'un déploiement

Réseau	Plan	Ce qui y circule	Nœuds raccordés	Exemple
Hors bande	Gestion	Pilotage matériel : IPMI, Redfish, console série	Carte BMC de chaque serveur, déploiement	VLAN 10 · 10.10.0.0/24
Provisionnement	Gestion	Démarrage PXE et installation du système	Tous, à l'installation	VLAN 11 · 10.11.0.0/24
Gestion · API interne	Contrôle	API internes, RabbitMQ, MariaDB, Memcached, OVSDB, SSH, supervision	Tous	VLAN 20 · 10.20.0.0/24
API publique	Contrôle	VIP externe en HTTPS : API, Horizon, consoles	Contrôleurs (HAProxy) uniquement	VLAN 30 · 192.0.2.0/24
Overlay	Données	Tunnels Geneve ou VXLAN transportant les réseaux de projets	Calcul, passerelles	VLAN 40 · 10.40.0.0/24 · MTU 9000
Stockage — accès	Données	Clients vers Ceph (réseau public), iSCSI, NFS	Calcul, contrôle, stockage	VLAN 50 · 10.50.0.0/24 · MTU 9000
Stockage — réplication	Données	Réplication et reconstruction entre OSD (réseau cluster)	Stockage uniquement	VLAN 51 · 10.51.0.0/24 · MTU 9000
Externe · fournisseur	Données	IP flottantes, réseaux fournisseurs en VLAN	Passerelles ; calcul si routage distribué	Trunk VLAN 100–199, sans adresse sur l'hôte

Le plan de contrôle circule sur deux réseaux : « gestion · API interne » pour tous les échanges entre composants, et « API publique » pour l'entrée des utilisateurs. Les réseaux hors bande et de provisionnement servent à administrer les machines elles-mêmes.

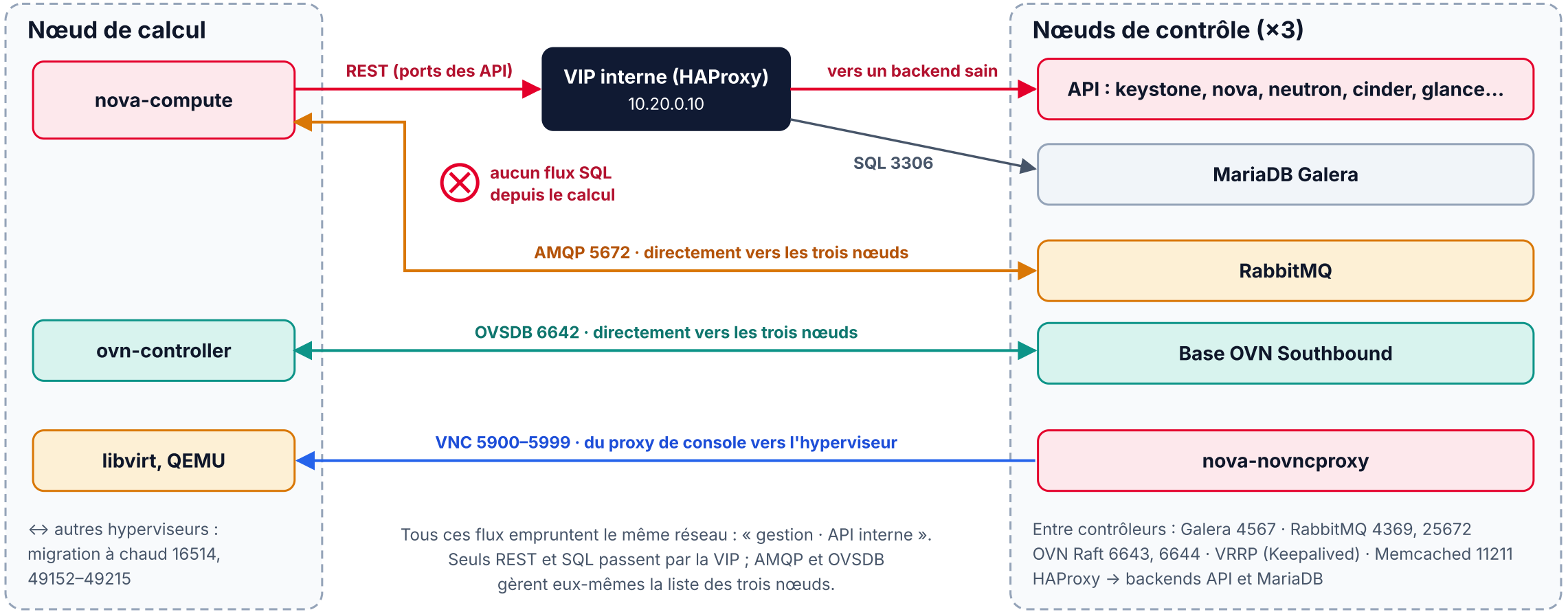
Les noms varient selon l'outil (management, internal API, tunnel, storage, storage management…), pas les rôles. Dans un lab, tout peut tenir sur un ou deux réseaux ; la séparation reste logique.

Quel nœud sur quel réseau

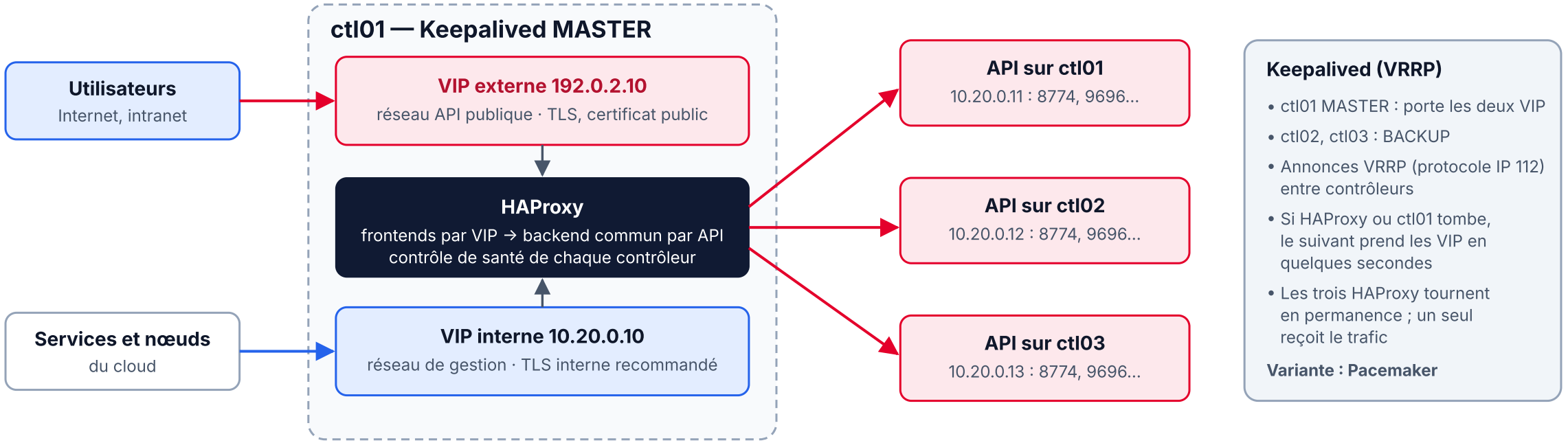


Un nœud n'est raccordé qu'aux réseaux nécessaires à son rôle : un nœud de calcul n'a **aucune patte** sur l'API publique ni sur la réplication du stockage, et ne rejoint l'externe (pointillés) que si les IP flottantes sont distribuées. Un nœud de stockage ne voit ni l'overlay ni l'extérieur. Seuls les contrôleurs portent l'API publique.

Les flux du plan de contrôle sur le réseau de gestion



VIP interne et VIP externe : deux entrées pour les mêmes API



VIP externe

Seule adresse du cloud visible des utilisateurs. Elle correspond aux points d'accès `public` du catalogue. On y applique certificat reconnu, limitation de débit, éventuel pare-feu applicatif.

VIP interne

Adresse utilisée par les services entre eux (points d'accès `internal`) et pour MariaDB. Jamais routée hors du réseau de gestion.

Pourquoi deux

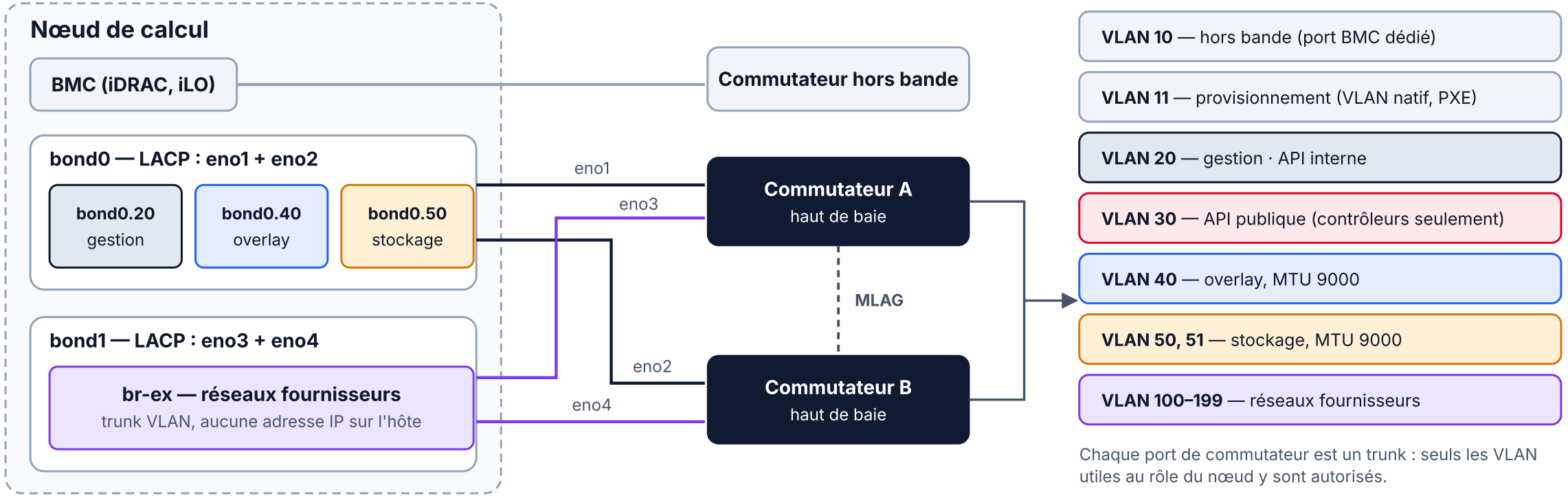
Le trafic interne ne sort pas vers le domaine public ; on peut filtrer ou couper l'accès externe sans casser le cloud ; les certificats et les règles diffèrent.

Sécuriser les flux du plan de contrôle

Isoler un réseau ne suffit pas : qui y prend pied voit passer jetons, mots de passe et messages RPC.

Flux	Chiffrement	Authentification	Remarque
REST, VIP externe	TLS obligatoire, terminé par HAProxy	Jeton Keystone	Certificat d'une autorité reconnue par les utilisateurs
REST, VIP interne	TLS interne, éventuellement jusqu'au backend	Jeton Keystone + jeton de service	Autorité de certification interne, diffusée à tous les nœuds
AMQP (RabbitMQ)	TLS, port 5671	Compte et mot de passe, par hôte virtuel	Un message RPC forgé équivaut à un ordre d'administrateur
SQL (MariaDB)	TLS client-serveur ; TLS pour la réplication Galera	Un compte par service, limité à son schéma	Jamais de compte de base sur les hyperviseurs
Memcached	Aucun par défaut	Aucune par défaut	Chiffrer les jetons mis en cache : `memcache_security_strategy = ENCRYPT`
OVSDB (OVN)	TLS avec certificats	Certificat client, contrôle d'accès par chassis	Un chassis ne doit modifier que ses propres lignes
libvirt, migration	TLS (16514) ou tunnel SSH	Certificats ou clés entre hyperviseurs	La mémoire d'une instance transite en clair sinon
Consoles VNC	TLS entre le proxy et l'hyperviseur (option)	Jeton de console à durée limitée	Le navigateur ne voit que le proxy, en HTTPS
Ceph	Mode sécurisé du protocole msgr2 (option)	Clés cephx, une par service client	Clé distincte pour Glance, Cinder, Nova
Tunnels Geneve	Aucun par défaut ; IPsec possible avec OVN	—	D'où l'isolement strict du réseau overlay

Du logique au physique : agrégats de liens et trunks VLAN



Chaque port de commutateur est un trunk : seuls les VLAN utiles au rôle du nœud y sont autorisés.

Redondance

Chaque agrégat (LACP 802.3ad) relie le serveur à deux commutateurs couplés (MLAG, vPC) : la perte d'un lien ou d'un commutateur est transparente.

Séparation

Le trafic des locataires (bond1) ne partage pas ses liens avec le contrôle et le stockage (bond0). Avec deux cartes seulement, tout passe par un seul agrégat et la séparation n'est que logique.

Hors bande

Le port BMC rejoint un commutateur distinct : il reste joignable quand le système, l'agrégat ou les commutateurs de production sont en panne.

Matrice de flux : la base des règles de pare-feu

Source	Destination	Port / protocole	Réseau	Objet
Utilisateurs	VIP externe	443, ou ports des API ; 6080	API publique	API, Horizon, consoles
Tous les nœuds	VIP interne	5000, 8774, 8775, 8778, 9696, 8776, 9292	Gestion	REST, métadonnées
Contrôleurs, cinder-volume	VIP interne	3306	Gestion	SQL
Contrôleurs, calcul, cinder-volume	Contrôleurs	5672 (ou 5671)	Gestion	AMQP
Calcul, passerelles	Contrôleurs	6642	Gestion	OVSDB, base Southbound
Contrôleurs	Contrôleurs	4567, 4568, 4444 ; 4369, 25672 ; 6643, 6644 ; 11211 ; VRRP	Gestion	Clusters et bascule
Contrôleurs	Calcul	5900–5999	Gestion	Consoles des instances
Calcul	Calcul	16514, 49152–49215	Gestion, ou dédié	Migration à chaud
Calcul, passerelles	Calcul, passerelles	UDP 6081 (ou 4789)	Overlay	Tunnels Geneve, VXLAN
Calcul, contrôleurs	Stockage	3300, 6789 ; 6800–7300 ; 3260	Stockage — accès	Ceph ; iSCSI (LVM)
Stockage	Stockage	6800–7300	Stockage — réplication	Réplication Ceph
Déploiement, bastion	Tous les nœuds	22	Gestion	Ansible, administration
Tous les nœuds	Serveurs NTP, DNS	UDP 123, 53	Gestion	NTP, DNS

Ce qui n'apparaît pas est interdit : en particulier, aucun flux des instances vers la gestion, aucun flux des utilisateurs ailleurs que vers la VIP externe.

MTU : le coût de l'encapsulation sur le réseau overlay

Trame émise par l'hyperviseur sur le réseau overlay (Geneve, IPv4)



MTU physique	Geneve	VXLAN
1500	1442	1450
9000	8942	8950

MTU annoncée aux instances d'un réseau de projet, en IPv4. Neutron la calcule et la diffuse par DHCP.

```
# neutron.conf
[DEFAULT]
global_physnet_mtu = 9000

# ml2_conf.ini
[ml2]
path_mtu = 9000
```

- **Symptôme d'une MTU incohérente :** le ping et l'ouverture SSH passent, les gros transferts et certaines négociations TLS se figent.
- La MTU doit être **identique de bout en bout** : cartes, agrégats, VLAN, ports des commutateurs.
- Test : `ping -M do -s 8972` entre deux hôtes du réseau overlay ou stockage.

> Le réseau de gestion reste en général à 1500 : il porte de petits messages. Les trames géantes profitent à l'overlay (pour offrir 1500 aux instances) et au stockage (débit de réplication).

Les flux de contrôle qui touchent aux instances

Quelques services doivent franchir la frontière entre le plan de contrôle et le domaine invité. Ce sont les passages les plus sensibles de l'architecture : chacun est étroit, filtré et authentifié.

Flux	Trajet	Ports	Garde-fou
Métadonnées	Instance → `169.254.169.254` → ovn-metadata-agent (sur l'hyperviseur) → nova-metadata	80 côté instance, 8775 côté API	L'agent ajoute l'identité de l'instance, signée par un secret partagé avec Nova
Consoles	Navigateur → VIP externe → nova-novncproxy → hyperviseur	6080, puis 5900–5999	Jeton de console à usage limité ; l'hyperviseur n'écoute que sur le réseau de gestion
Octavia	Contrôleurs Octavia ↔ amphores, par un réseau de gestion dédié (`lb-mgmt-net`)	TCP 9443, UDP 5555	TLS mutuel ; réseau distinct du réseau de gestion du cloud
Ironic	ironic-conductor ↔ serveurs physiques : réseaux de provisionnement, de nettoyage, BMC	PXE, HTTP, IPMI ou Redfish	Le serveur ne rejoint le réseau du projet qu'une fois déployé
Migration à chaud	Hyperviseur source → hyperviseur cible	16514, 49152–49215	Réseau dédié possible (`live_migration_inbound_addr`), TLS
DHCP et DNS des projets	Instance → service fourni localement par OVN sur l'hyperviseur	UDP 67, 53	Réponses générées par des flux OVS : aucun démon exposé aux instances

Principe commun : l'instance ne dialogue jamais directement avec un service du plan de contrôle. Un relais situé sur l'hyperviseur ou sur un réseau dédié s'interpose, restreint la demande et l'authentifie avant de la transmettre.

 Équivalents Sunbeam : module 3, slide 5

Traduction en configuration

```
# /etc/kolla/globals.yml (Kolla-Ansible)
network_interface: "bond0.20"       # gestion · API interne
kolla_internal_vip_address: "10.20.0.10"
kolla_internal_fqdn: "int.cloud.lan"

kolla_external_vip_interface: "bond0.30"
kolla_external_vip_address: "192.0.2.10"
kolla_external_fqdn: "cloud.example.org"

tunnel_interface: "bond0.40"        # overlay Geneve
storage_interface: "bond0.50"       # accès Ceph, iSCSI
migration_interface: "bond0.20"
neutron_external_interface: "bond1" # br-ex, sans IP

kolla_enable_tls_external: "yes"
kolla_enable_tls_internal: "yes"
```

Réseau	Kolla-Ansible	OpenStack-Ansible
Gestion	`network_interface`	`br-mgmt`
Overlay	`tunnel_interface`	`br-vxlan`
Stockage	`storage_interface`	`br-storage`
Fournisseur	`neutron_external_interface`	`br-vlan`

```
# Résultat sur un nœud de calcul : nova.conf
[DEFAULT]
my_ip = 10.20.0.31
transport_url = rabbit://…@10.20.0.11:5672,…@10.20.0.12:5672,…
[vnc]
server_listen = 10.20.0.31
server_proxyclient_address = 10.20.0.31
novncproxy_base_url = https://cloud.example.org:6080/vnc_lite.html
[libvirt]
live_migration_inbound_addr = 10.20.0.31
[neutron]
valid_interfaces = internal
```

```
# Résultat sur le même nœud : enregistrement OVN
$ ovs-vsctl get open . external_ids
{ovn-encap-ip="10.40.0.31", ovn-encap-type=geneve,
 ovn-remote="ssl:10.20.0.11:6642,ssl:10.20.0.12:6642,ssl:10.20.0.13:6642",
 system-id="cmp01"}
```

Lecture : le contrôle (RabbitMQ, OVSDB, consoles) utilise les adresses `10.20.0.x` du réseau de gestion ; les tunnels partent de `10.40.0.31`, sur le réseau overlay. Une même machine, deux réseaux, deux usages.

SYNTHÈSE

À retenir

1 · Un rôle = des processus + des réseaux

Contrôle : API, base, bus, VIP. Calcul : nova-compute, libvirt, ovn-controller. Passerelle : sortie externe. Stockage : Ceph ou Swift.

2 · Contrôle et données sont découplés

Trois contrôleurs pour le quorum. Leur perte fige le cloud sans l'éteindre : instances, réseaux et volumes continuent.

3 · Quatre protocoles internes

REST entre services, AMQP dans un service, SQL vers la base, OVSDB pour OVN. Chaque panne se rattache à l'un d'eux.

4 · Le bus transporte des ordres

call (avec réponse), cast (sans), fanout (à tous). Les files sont créées par les services ; une file qui grossit désigne le coupable.

5 · L'hyperviseur n'a pas la base

nova-compute passe par nova-conductor. REST et SQL passent par la VIP interne ; AMQP et OVSDB visent directement les trois nœuds.

6 · Un réseau par famille de flux

Gestion · API interne, API publique, overlay, stockage, externe, hors bande. Un nœud ne rejoint que ceux de son rôle.

Question à se poser devant un déploiement	Où trouver la réponse
Quels processus tournent sur ce nœud ?	Inventaire de l'outil de déploiement ; listes de services Nova, Cinder, Neutron
Par où passent les appels entre services ?	`openstack endpoint list` : interfaces `internal` et `public`
Le bus est-il sain ?	`rabbitmqctl cluster_status`, `rabbitmqctl list_queues name messages consumers`
La base a-t-elle son quorum ?	`SHOW STATUS LIKE 'wsrep_cluster_size'`
Chaque chassis est-il enregistré ?	`ovn-sbctl show`, `openstack network agent list`
Où une opération s'est-elle arrêtée ?	Identifiant `req-…` recherché dans les journaux centralisés